Multi-Cloud Simulation Platform

Cloud Simulator in the Style of Cisco Packet Tracer

Complete Developer & QA Blueprint

Vision | Architecture | Theory Modules | Practical Simulation | Use Cases
Testing Stages | Test Cases | Roadmap | Risks

Azure - AWS - GCP (optional Oracle / IBM / Alibaba)
Free of cost - Offline PWA / APK release

Table of Contents

1. Vision & Goals **4**

1.1 Vision . . . 4

1.2 Objectives . . . 4

1.3 Scope . . . 4

2. Architecture **4**

2.1 Technology Stack . . . 4

2.2 Logical Architecture . . . 5

2.3 Component Responsibilities . . . 5

2.4 Common Resource Model . . . 5

2.5 Database Design (PostgreSQL) . . . 6

2.6 API Outline (FastAPI) . . . 6

2.7 Offline Strategy (PWA / APK) . . . 6

2.8 Non-Functional Requirements . . . 7

3. Theory Modules **8**

3.1 Suggested Module Structure . . . 8

3.2 Theory-to-Practical Mapping . . . 8

4. Practical Simulation **9**

4.1 Challenge Labs (Debug and Fix) . . . 9

4.2 Feedback Model . . . 9

4.3 Simulation Rules Summary . . . 9

5. Use Cases **10**

5.1 Actors . . . 10

5.2 Use Case List . . . 10

5.3 Detailed Use Cases . . . 10

6. Testing Strategy & Stages **12**

6.1 Unit Testing . . . 12

6.2 Integration Testing . . . 12

6.3 System Testing . . . 12

6.4 UAT (User Acceptance Testing) . . . 12

6.5 Security & Performance . . . 12

6.6 Deployment Testing . . . 12

6.7 Stage Summary . . . 13

7. Test Cases **14**

7.1 Sample Test Cases (from blueprint) . . . 14

7.2 Detailed Test Cases . . . 14

7.3 Additional Domain Test Cases . . . 17

7.4 Test Case Template (for QA to add more) . . . 17

7.5 Traceability: Use Cases to Tests . . . 17

8. Sample Backend Python Code **18**

8.1 CIDR Overlap Check . . . 18

8.2 Matching Unit Tests (pytest) . . . 18

9. Roadmap **19**

9.1 Build Steps per Phase . . . 19

9.2 Testing per Phase . . . 19

10. Implementation Guide - Stage by Stage **20**

10.1 How to Use This Guide . . . 20
10.2 Stage 0 - Project Set-up . . . 20
10.3 Stage 1 - Phase 1: AZ-900 Foundation . . . 22
10.4 Stage 2 - Phase 2: AZ-104 Administrator . . . 29
10.5 Stage 3 - Phase 3: AWS + GCP Integration . . . 37
10.6 Stage 4 - Phase 4: Specialty Modules . . . 42
10.7 Stage 5 - Final: Offline Release + Gamification . . . 44
10.8 Stage Checklist Summary . . . 46
11. Exam Preparation & Workflow Diagrams 47
11.1 Exam Preparation Roadmap . . . 47
11.2 Study Plan Checklist . . . 47
11.3 Bug Handling Flow . . . 47
11.4 VM Lifecycle Flow . . . 48
11.5 Multi-Cloud Challenge Lab (Worked Example) . . . 48
11.6 Challenge Lab Checklist . . . 49
12. Risks & Mitigations 49
13. Final Note 50

1. Vision & Goals

The platform is a cloud-simulation learning environment that works like Cisco Packet Tracer, but for public cloud. Learners design, build, break and fix cloud infrastructure on a visual canvas without ever creating a real cloud account or paying a bill.

1.1 Vision

- A Cisco Packet Tracer style cloud simulator.
- Covers Azure, AWS and GCP, with optional Oracle, IBM and Alibaba Cloud.
- Theory and Practical in one place - learners read a concept and immediately practise it.
- Free of cost - released as an offline-capable PWA and an Android APK.

1.2 Objectives

- Teach certification-aligned content (Azure Fundamentals to Expert and Specialty, plus AWS and GCP essentials).
- Provide a safe sandbox: no real resources, no billing, no risk of exposing credentials.
- Give instant, accurate feedback on mistakes such as CIDR overlaps, wrong routes, open firewalls and over-permissive IAM.
- Run fully offline on low-end devices, with all data stored locally.
- Give trainers a way to author labs, run classroom sessions (for example 20 students) and track progress.

1.3 Scope

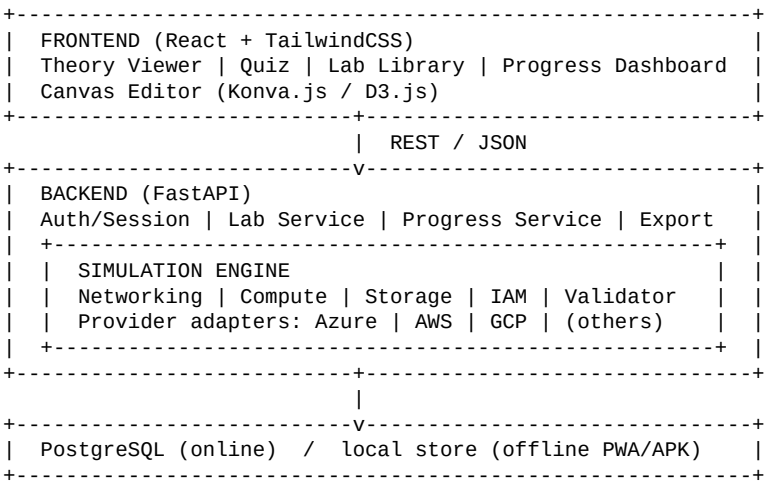
In scope	Out of scope (initial release)
Networking, Compute, Storage and IAM simulation for Azure, AWS and GCP	Real cloud provisioning or live billing integration
Theory modules and quizzes mapped to certifications	Pixel-perfect replicas of vendor portals
Debug/fix challenges, lab export as JSON/YAML	Simulation of every service in every cloud (services are added phase by phase)
Offline PWA and APK, local data storage	Paid tiers or commercial hosting

2. Architecture

2.1 Technology Stack

Layer	Technology	Responsibility
Frontend	React + TailwindCSS	UI shell, theory pages, quizzes, dashboards, lab library
Canvas	Konva.js / D3.js	Drag-and-drop topology editor: VNets/VPCs, subnets, VMs, buckets, links
Backend	Python (FastAPI)	REST API, lab validation, progress tracking, import/export
Simulation Engine	Python module (inside backend, also portable to client)	CIDR maths, routing, firewall/NSG/Security Group evaluation, IAM evaluation, resource lifecycle
Database	PostgreSQL	Lab configurations, user progress, quiz results (SQLite/IndexedDB used for the offline build)
Packaging	PWA (service worker) + APK wrapper	Offline install on browsers and Android

2.2 Logical Architecture



2.3 Component Responsibilities

Component	Responsibility
Canvas Editor	Draws resources and links, captures user actions, sends the topology graph to the engine for validation.
Lab Service	Loads lab definitions (starting state, goals, hints), saves user lab state, handles JSON/YAML export and import.
Progress Service	Stores quiz scores, lab completion, challenge status, badges.
Simulation Engine	Pure logic layer. Given a topology and an action it returns a new state plus validation messages. No network calls, so it is easy to unit-test and to run offline.
Provider Adapters	Map provider vocabulary (VNet/NSG/Blob, VPC/Security Group/S3, VPC/Firewall Rule/Bucket) to the common internal resource model.
Validator / Feedback	Produces human-readable errors, warnings and hints (for example 'Subnet 10.0.1.0/24 overlaps 10.0.1.0/25').
Storage Layer	PostgreSQL for hosted use; local storage (IndexedDB/SQLite) for the offline PWA/APK so nothing leaves the device.

2.4 Common Resource Model

All providers are normalised to one internal model so the engine has one code path per feature:

- **Network** (VNet / VPC) - address space, list of subnets, peerings.
- **Subnet** - CIDR, parent network, route table, firewall/NSG association.
- **RouteTable** - routes (destination CIDR, next hop).
- **FirewallRule** - priority, direction, protocol, port range, source/destination, allow/deny.
- **Instance** (VM / EC2 / Compute Engine) - size, subnet, state (creating, running, stopped, deleted), logs.
- **StorageContainer** (Blob container / S3 bucket / GCS bucket) - objects, quota, access level, public-access flag.
- **Identity** and **Role/Policy** - principals, role assignments, allow/deny statements, scope.

2.5 Database Design (PostgreSQL)

Table	Key columns	Purpose
users	id, name, role (student/trainer/admin), created_at	Learner and trainer accounts (local profile in offline mode)
courses / modules	id, certification_code, title, level, phase	Theory modules such as AZ-900, AZ-104
lessons	id, module_id, content, order	Theory content
quizzes / quiz_results	id, module_id, question set, score, user_id, taken_at	Quiz definitions and results
labs	id, module_id, provider, title, difficulty, goal_spec (JSON)	Lab definitions and success criteria
lab_configs	id, lab_id, user_id, topology (JSON), version, updated_at	Saved user topologies and exports
lab_progress	id, user_id, lab_id, status, attempts, completed_at	Progress tracking
challenges	id, lab_id, broken_state (JSON), fix_criteria (JSON)	Debug/fix challenges
validation_logs	id, lab_config_id, rule, result, message, latency_ms	Feedback history and performance measurement

2.6 API Outline (FastAPI)

Method	Endpoint	Description
GET	/api/modules	List theory modules and certification tracks
GET	/api/modules/{id}/lessons	Lessons for a module
POST	/api/quizzes/{id}/submit	Submit quiz answers, get score
GET	/api/labs	Lab library (filter by provider/level)
POST	/api/labs/{id}/validate	Validate a topology against the lab goals (target under 200 ms)
POST	/api/sim/network/check-cidr	CIDR overlap check
POST	/api/sim/network/route	Route / reachability simulation
POST	/api/sim/firewall/evaluate	NSG / Security Group / firewall rule evaluation
POST	/api/sim/iam/evaluate	IAM allow/deny evaluation
POST	/api/sim/compute/{action}	Create / start / stop / delete instance
POST	/api/sim/storage/{action}	Upload / delete / set access on storage
GET / PUT	/api/lab-configs/{id}	Load or save a user's lab state
GET	/api/lab-configs/{id}/export?format=json|yaml	Export lab (multi-cloud export)
POST	/api/lab-configs/import	Import lab from JSON/YAML
GET	/api/progress/{user_id}	Progress, badges, completion tracking

2.7 Offline Strategy (PWA / APK)

- Service worker caches the app shell, theory content, lab definitions and quiz banks.
- The simulation engine is portable so validation can run on-device with no server (keeps latency low and cost at zero).
- All learner data is stored locally; there are no cloud credentials and no billing.
- APK is a wrapper around the same PWA build to keep a single code base.
- Optional sync of lab exports (JSON/YAML) between devices by file share.

2.8 Non-Functional Requirements

Area	Requirement
Performance	Lab validation latency under 200 ms; canvas stays responsive with typical lab sizes.
Security	Unauthorized access blocked; no secrets or real credentials stored; local data protected by the OS/app sandbox.
Cost	Zero billing - nothing is ever provisioned in a real cloud.
Compatibility	Multiple Android versions, major browsers, low-end devices.
Reliability	Deterministic engine: same input always gives same output, so tests are repeatable.
Maintainability	Provider adapters isolated so a new cloud (Oracle, IBM, Alibaba) is added without touching core logic.
Usability	Clear, actionable error messages and hints.

3. Theory Modules

Theory content is organised by certification level so a learner can follow a path and then practise the same topics in the lab.

Level	Certifications / Topics
Fundamentals	AZ-900 (Azure Fundamentals), AI-901, DP-900 (Data Fundamentals), SC-900 (Security, Compliance, Identity Fundamentals)
Associate	AZ-104 (Administrator), AZ-204 (Developer), AI-200, AZ-500 (Security Engineer), SC-500, DP-203 (Data Engineer), DP-300 (Database Administrator), PL-300 (Power BI Data Analyst), AZ-700 (Network Engineer)
Expert	AZ-305 (Solutions Architect), AZ-400 (DevOps Engineer), SC-100 (Cybersecurity Architect)
Specialty	AZ-140 (Azure Virtual Desktop), AZ-120 (SAP on Azure), DP-420 (Cosmos DB Developer)
Extras	AWS: VPC, EC2, S3, IAM. GCP: VPC, Compute Engine, Buckets.

3.1 Suggested Module Structure

- Concept lessons (short text + diagrams).
- 'Try it in the lab' link from each lesson to a related practical.
- End-of-module quiz with explanation for every answer.
- Progress badge when the quiz and linked lab are both completed.

3.2 Theory-to-Practical Mapping

Theory area	Related practical lab
Azure / AWS / GCP networking (AZ-900, AZ-104, AZ-700)	VNet/VPC creation, subnetting, routing, peering, firewall rules
Compute (AZ-900, AZ-104, AZ-204)	VM / EC2 / Compute Engine creation and lifecycle
Storage (AZ-900, AZ-104, DP-900)	Blob / S3 / Bucket upload, quota, public-access control
Identity and security (SC-900, AZ-500, SC-500, SC-100)	IAM roles, policies, allow/deny, least-privilege challenges
Architecture (AZ-305)	Multi-cloud connectivity design and review labs
DevOps (AZ-400)	Lab export as JSON/YAML, infrastructure-as-code style exercises

4. Practical Simulation

Domain	What is simulated	Provider terms
Networking	VNet/VPC, Subnet, Routing, Firewall rules, peering	Azure VNet + NSG; AWS VPC + Security Group / NACL; GCP VPC + Firewall Rules
Compute	VM/EC2/Instance creation, start, stop, delete, state and logs	Azure VM; AWS EC2; GCP Compute Engine
Storage	Blob/S3/Bucket, upload, quota, access level, public access	Azure Blob Storage; AWS S3; GCP Cloud Storage buckets
IAM	Roles, Policies, Access control, allow/deny evaluation	Azure RBAC; AWS IAM; GCP IAM

4.1 Challenge Labs (Debug and Fix)

- **Fix subnet overlap** - learner receives a broken VNet/VPC with overlapping CIDRs and must re-plan the address space.
- **Debug broken infrastructure** - a VM cannot reach storage or the internet; learner finds the missing route or blocking firewall rule.
- **Secure IAM** - an over-permissive role is granted; learner applies least privilege and verifies a deny works.
- **Multi-cloud connectivity** - connect an Azure VNet to an AWS VPC without overlapping ranges.
- **Lock down storage** - stop a public bucket/blob and confirm anonymous access is blocked.

4.2 Feedback Model

- Every action returns Pass / Warning / Error with a short explanation.
- Optional hint levels (nudge, more detail, full answer) to support learning.
- Lab completion is decided by goal criteria stored with the lab (goal_spec).

4.3 Simulation Rules Summary

Rule	Behaviour
CIDR validation	Reject invalid CIDR notation and any subnet outside the parent range; flag overlaps between subnets and between peered networks.
Routing	Longest-prefix match; next hop types (local, peering, internet, virtual appliance, none).
Firewall / NSG / SG	Rules evaluated by priority; first match wins where the provider works that way; default deny/allow follows the provider.
IAM	Explicit deny overrides allow; no matching allow means deny; cross-cloud role conflicts reported as a warning/error.
Compute lifecycle	creating -> running -> stopped -> deleted; each transition writes a log entry.
Storage	Quota is enforced on upload; public access is governed by bucket/account settings plus IAM.

5. Use Cases

5.1 Actors

Actor	Description
Student / Learner	Reads theory, takes quizzes, builds and fixes labs, tracks progress, works offline.
Trainer / Instructor	Assigns labs, runs a class session (for example 20 students), reviews progress and feedback.
Lab Author / Admin	Creates and maintains lab definitions, challenges, quiz banks and module content.
Developer / QA	Extends the engine and adapters, writes and runs automated tests, validates releases.

5.2 Use Case List

ID	Use case	Actor	Summary
UC-01	Study a theory module	Student	Open a certification module (for example AZ-900), read lessons, follow the link to a lab.
UC-02	Take a quiz	Student	Answer module quiz, see score and explanations, result stored in progress.
UC-03	Build a network	Student	Create VNet/VPC, add subnets, routes and firewall rules on the canvas.
UC-04	Create a VM / EC2 / Instance	Student	Place a compute resource in a subnet, start and stop it, view the log.
UC-05	Manage storage	Student	Create a Blob/S3/Bucket, upload objects, set access and check quota.
UC-06	Configure IAM	Student	Assign roles, write allow/deny policies, test access.
UC-07	Fix a broken infrastructure challenge	Student	Diagnose and repair a pre-broken lab, then submit for validation.
UC-08	Connect clouds	Student	Peer/connect Azure VNet with AWS VPC; validate routing and CIDR.
UC-09	Export / import a lab	Student, Trainer	Save the lab as JSON or YAML; load it on another device.
UC-10	Track progress	Student	View completed modules, quizzes, labs and challenges.
UC-11	Use the app offline	Student	Install PWA/APK, use all content and labs without internet.
UC-12	Run a class session	Trainer	Distribute labs to about 20 students and review results.
UC-13	Author a lab or challenge	Lab Author	Define starting topology, goal criteria, hints and expected feedback.
UC-14	Add a new cloud provider	Developer	Implement a provider adapter (Oracle / IBM / Alibaba) against the common model.
UC-15	Gamification	Student	Earn badges/points for completions (final roadmap phase).

5.3 Detailed Use Cases

UC-03 - Build a network	
Actor	Student
Preconditions	Lab opened; provider selected (Azure, AWS or GCP).
Main flow	1) Add a VNet/VPC with an address space. 2) Add subnets with CIDR ranges. 3) Attach a route table. 4) Add firewall/NSG/security-group rules. 5) Click Validate. 6) Engine returns results.
Alternate / error flow	Overlapping or invalid CIDR -> error message with the conflicting ranges. Rule blocks required traffic -> warning naming the rule.
Postconditions	Valid topology saved to lab_configs; progress updated.

UC-07 - Fix a broken infrastructure challenge	
Actor	Student
Preconditions	Challenge lab loaded with a broken starting state.
Main flow	1) Inspect resources and logs. 2) Identify fault (overlap, missing route, blocking rule, bad IAM). 3) Modify configuration. 4) Submit for validation.
Alternate / error flow	Goal not met -> feedback and optional hint; attempts counter incremented.
Postconditions	Challenge marked complete when all fix criteria pass.

UC-08 - Connect clouds (Azure VNet and AWS VPC)	
Actor	Student
Preconditions	One Azure VNet and one AWS VPC exist in the lab.
Main flow	1) Choose the multi-cloud connection. 2) Engine checks CIDR overlap. 3) Routes are added on both sides. 4) Firewall rules are checked. 5) Reachability test is run.
Alternate / error flow	Overlap -> connection refused with explanation. Missing route on either side -> reachability fails with the missing hop shown.
Postconditions	Connectivity state stored; lab can be exported as JSON/YAML.

UC-11 - Use the app offline	
Actor	Student
Preconditions	PWA or APK installed once.
Main flow	1) Open the app without internet. 2) Read theory, take quizzes, run labs. 3) Validation runs on-device. 4) Progress stored locally.
Alternate / error flow	Feature that needs network (none by design) -> clear 'not available offline' message.
Postconditions	All data remains on the device; no billing or cloud calls made.

UC-12 - Run a class session	
Actor	Trainer
Preconditions	Trainer has the lab set; about 20 students have the app.
Main flow	1) Trainer selects a lab. 2) Students build it. 3) Students export results (JSON/YAML) or trainer views progress. 4) Trainer reviews common errors.
Alternate / error flow	Student device is low-end/slow -> lab still completes within the latency target.
Postconditions	Class results available for review and UAT feedback.

6. Testing Strategy & Stages

Testing follows six stages. Each stage lists what is verified, the exit criteria, and the owner. Test IDs in section 7 map back to these stages.

6.1 Unit Testing

- CIDR overlap validation.
- Routing table simulation.
- IAM role allow/deny logic.
- VM/EC2 instance creation.

Exit criteria: all engine functions covered; deterministic results; edge cases (/0, /32, invalid CIDR) handled. **Owner:** Developers.

6.2 Integration Testing

- Azure VNet to AWS VPC connectivity.
- Storage + IAM combined access.
- Multi-cloud lab export (JSON/YAML).

Exit criteria: modules work together through the API; export then import reproduces the same topology. **Owner:** Developers + QA.

6.3 System Testing

- End-to-end lab build (20 students).
- Offline mode validation (PWA/APK).
- Error feedback correctness.

Exit criteria: full flow (theory -> lab -> validation -> progress) works for the whole class load, online and offline. **Owner:** QA.

6.4 UAT (User Acceptance Testing)

- Students run labs and give feedback on clarity.
- Quiz and challenge completion tracking.

Exit criteria: learners can finish labs without trainer help; feedback messages rated clear; completion data correct. **Owner:** Trainers + students + QA.

6.5 Security & Performance

- Unauthorized access blocked.
- Latency under 200 ms for lab validation.
- Data stored locally (no billing).

Exit criteria: no access-control bypass; 95th-percentile validation under 200 ms; no external calls or billing exposure. **Owner:** QA + Developers.

6.6 Deployment Testing

- APK install across Android versions.
- PWA install across browsers.
- Low-end device compatibility.

Exit criteria: install, launch, update and offline use verified on the supported device/browser matrix. **Owner:** QA.

6.7 Stage Summary

Stage	Focus	Environment	Automation
Unit	Engine functions	Developer machine / CI	Fully automated (pytest)
Integration	Module + API interaction	CI with test DB	Automated (pytest + API tests)
System	End-to-end flows, class load, offline	Staging build	Mostly automated (E2E) + manual
UAT	Clarity, completion tracking	Real learners	Manual + feedback forms
Security & Performance	Access control, latency, local data	Staging / device	Automated load + manual review
Deployment	APK/PWA install, low-end devices	Device lab / emulators	Manual + emulator scripts

7. Test Cases

7.1 Sample Test Cases (from blueprint)

These are the baseline cases defined in the blueprint. Each has an ID for traceability.

ID	Area	Case	Expected result
NET-01	Networking	Create VNet, then create a subnet that overlaps an existing one	Subnet overlap error shown
NET-02	Networking	Add NSG rule	Traffic allow/deny validated correctly
NET-03	Networking	VNet peering	Routing table updated
CMP-01	Compute	VM creation	Instance state stored
CMP-02	Compute	EC2 start/stop	Log entry created
CMP-03	Compute	GCP Compute Engine instance	Firewall applied
STO-01	Storage	Blob upload	Storage quota validated
STO-02	Storage	S3 bucket access with IAM deny	Access denied as expected
STO-03	Storage	GCP bucket public access	Public access blocked
IAM-01	IAM	Role assignment	Allow policy validated
IAM-02	IAM	Deny policy	Access blocked
IAM-03	IAM	Multi-cloud role conflict	Error feedback shown

7.2 Detailed Test Cases

Priority: P1 = critical, P2 = important, P3 = nice to have.

7.2.1 Unit Test Cases

ID	Test	Steps / input	Expected result	Pri
UT-01	CIDR overlap detected	Subnets 10.0.1.0/24 and 10.0.1.0/25	Overlap reported with both ranges named	P1
UT-02	Non-overlapping CIDR accepted	10.0.1.0/24 and 10.0.2.0/24	No error	P1
UT-03	Invalid CIDR rejected	10.0.0.0/33, 300.1.1.1/24, empty string	Validation error for each	P1
UT-04	Subnet outside network range	VNet 10.0.0.0/16, subnet 192.168.1.0/24	Error: subnet not within address space	P1
UT-05	Longest-prefix route match	Routes 10.0.0.0/16 and 10.0.1.0/24; destination 10.0.1.5	The /24 route is chosen	P1
UT-06	No matching route	Destination not covered by any route	Traffic dropped; explanation returned	P2
UT-07	IAM allow	Role with read on bucket; user reads	Allowed	P1
UT-08	IAM explicit deny overrides allow	Allow and deny both match	Denied	P1
UT-09	IAM implicit deny	No policy matches	Denied	P1
UT-10	Instance creation	Create VM/EC2 in valid subnet	Instance stored with state creating -> running	P1
UT-11	Instance in missing subnet	Create VM with no subnet	Error returned	P2
UT-12	Instance lifecycle logging	Start then stop instance	Two log entries with timestamps	P2
UT-13	Firewall priority order	Allow (priority 200) and deny (priority 100) for the same flow	Lower number wins where provider defines it so; result matches provider rule	P1
UT-14	Storage quota	Upload exceeding quota	Upload rejected	P2

7.2.2 Integration Test Cases

ID	Test	Steps / input	Expected result	Pri
IT-01	Azure VNet to AWS VPC connectivity	Non-overlapping ranges; add routes both sides; run reachability	Reachable	P1
IT-02	Azure to AWS with overlapping CIDR	Same 10.0.0.0/16 on both	Connection refused with overlap message	P1
IT-03	Missing return route	Route only on one side	Reachability fails; missing hop identified	P2
IT-04	Storage + IAM combined	Bucket private; user has no role	Access denied	P1
IT-05	Storage + IAM grant	Assign read role, retry	Access allowed	P1
IT-06	Export lab as JSON	Build multi-cloud lab, export	Valid JSON containing all resources	P1
IT-07	Export lab as YAML	Same lab, YAML export	Valid YAML equivalent to JSON	P2
IT-08	Import restores lab	Export then import on clean profile	Identical topology on canvas	P1
IT-09	API to DB persistence	Save lab config then reload	Same state loaded (version incremented)	P2
IT-10	Quiz result to progress	Submit quiz	Score stored and visible on dashboard	P2

7.2.3 System Test Cases

ID	Test	Steps / input	Expected result	Pri
ST-01	End-to-end lab build	Theory -> lab -> validate -> progress saved	Whole flow completes with no error	P1
ST-02	20 students concurrently	20 sessions build the same lab	All validations succeed; latency target met	P1
ST-03	Offline mode	Install, go offline, run labs and quizzes	Everything works; data saved locally	P1
ST-04	Error feedback correctness	Trigger each known error type	Message is accurate and names the faulty resource	P1
ST-05	Challenge completion	Fix broken lab and submit	Marked complete; badge/progress updated	P2
ST-06	Lab reset	Reset lab after changes	Returns to original starting state	P3
ST-07	Provider switch	Run the same lab on Azure, AWS and GCP terminology	Equivalent behaviour with correct provider names	P2

7.2.4 UAT Test Cases

ID	Test	Steps / input	Expected result	Pri
UAT-01	Lab clarity	Student completes a lab without help	Finishes; rates instructions clear	P1
UAT-02	Feedback usefulness	Student triggers an error	Student understands what to fix from the message	P1
UAT-03	Quiz tracking	Complete quiz	Score and attempt visible	P2
UAT-04	Challenge tracking	Complete a challenge	Completion recorded and shown	P2
UAT-05	Trainer review	Trainer reviews class results	Progress readable for all students	P2

7.2.5 Security & Performance Test Cases

ID	Test	Steps / input	Expected result	Pri
SP-01	Unauthorized access blocked	Access another user's lab config / admin function	Denied	P1
SP-02	Input injection	Malformed JSON/YAML import, oversized payload	Rejected safely; no crash	P1
SP-03	Validation latency	Run 100 validations on a typical lab	Under 200 ms (95th percentile)	P1
SP-04	Local data only	Monitor network during use	No external calls; no billing data	P1
SP-05	Data persistence	Close and reopen app	Progress and labs preserved	P2
SP-06	Large topology	Lab with many subnets and instances	UI responsive; validation acceptable	P2

7.2.6 Deployment Test Cases

ID	Test	Steps / input	Expected result	Pri
DT-01	APK install	Install on several Android versions	Installs and launches	P1
DT-02	PWA install	Install on major browsers	Installable; runs standalone	P1
DT-03	Low-end device	Run typical lab on a low-RAM device	Usable; no crash	P1
DT-04	Update	Release new version	App updates; user data intact	P2
DT-05	Offline first launch	Install, then launch with no network	Cached content available	P2

7.3 Additional Domain Test Cases

ID	Area	Case	Expected result
NET-04	Networking	Subnet CIDR equals VNet range exactly	Accepted (or provider-specific note on reserved addresses)
NET-05	Networking	Peering between overlapping VNets	Refused with overlap error
NET-06	Networking	NSG rule denies port 22, then SSH test	SSH blocked
NET-07	Networking	AWS Security Group allows 443 only	Only 443 reachable
CMP-04	Compute	Stop already stopped instance	Clear message, state unchanged
CMP-05	Compute	Delete running instance	State becomes deleted; log written
STO-04	Storage	Azure Blob with anonymous access on and IAM deny	Behaviour follows provider precedence; result explained
STO-05	Storage	Delete object with read-only role	Denied
IAM-04	IAM	Least-privilege challenge fixed	Challenge passes
IAM-05	IAM	Scope: role at resource-group level vs resource level	Inheritance applied correctly

7.4 Test Case Template (for QA to add more)

Field	Description
Test ID	Unique ID (prefix by area: NET, CMP, STO, IAM, UT, IT, ST, UAT, SP, DT)
Title	Short name of the scenario
Preconditions	State needed before starting
Steps	Numbered actions
Expected result	Observable outcome
Actual result / Status	Pass / Fail / Blocked, with notes
Priority	P1 / P2 / P3
Linked requirement	Use case ID (UC-xx) or roadmap phase

7.5 Traceability: Use Cases to Tests

Use case	Covered by tests
UC-01, UC-02	IT-10, UAT-03
UC-03 Build a network	UT-01 to UT-06, UT-13, NET-01 to NET-07
UC-04 Compute	UT-10 to UT-12, CMP-01 to CMP-05
UC-05 Storage	UT-14, STO-01 to STO-05, IT-04, IT-05
UC-06 IAM	UT-07 to UT-09, IAM-01 to IAM-05
UC-07 Challenge	ST-05, UAT-04, IAM-04
UC-08 Connect clouds	IT-01 to IT-03
UC-09 Export / import	IT-06 to IT-08
UC-10 Progress	IT-09, IT-10, SP-05
UC-11 Offline	ST-03, DT-05, SP-04
UC-12 Class session	ST-02, UAT-05
UC-13 to UC-15	Verified in later phases (authoring tools, adapter tests, gamification)

8. Sample Backend Python Code

Sample code for the Simulation Engine. It uses only the Python standard library (ipaddress), so it runs offline and has no extra dependencies.

8.1 CIDR Overlap Check

```
import ipaddress

def check_cidr_overlap(cidr1, cidr2):
    net1 = ipaddress.ip_network(cidr1)
    net2 = ipaddress.ip_network(cidr2)
    return net1.overlaps(net2)

# Example
print(check_cidr_overlap("192.168.1.0/24", "192.168.1.128/25"))  # True
```

How it works: ip_network() parses each CIDR and raises ValueError for invalid input (for example /33 or a bad address). overlaps() returns True when one range contains or intersects the other. In the example, 192.168.1.128/25 lies inside 192.168.1.0/24, so the result is True.

Related test cases: UT-01, UT-02, UT-03, NET-01, IT-02. **Note:** a CIDR with host bits set (for example 192.168.1.5/24) also raises ValueError unless strict=False is passed to ip_network(); decide one behaviour for the whole engine and test it.

8.2 Matching Unit Tests (pytest)

```
import pytest
from cidr import check_cidr_overlap

def test_overlap_detected():          # UT-01
    assert check_cidr_overlap("192.168.1.0/24", "192.168.1.128/25") is True

def test_no_overlap():                # UT-02
    assert check_cidr_overlap("10.0.1.0/24", "10.0.2.0/24") is False

def test_invalid_cidr_rejected():     # UT-03
    with pytest.raises(ValueError):
        check_cidr_overlap("10.0.0.0/33", "10.0.1.0/24")
```

9. Roadmap

Phase	Deliverable	Content
Phase 1	AZ-900 module (Fundamentals)	Theory + quiz + first labs (basic networking, VM, storage, IAM concepts)
Phase 2	AZ-104 module (Administrator)	Deeper networking, compute, storage, IAM labs and challenges
Phase 3	AWS + GCP integration	VPC, EC2, S3, IAM; GCP VPC, Compute Engine, Buckets; provider adapters
Phase 4	Specialty modules	AZ-305, AZ-400, AZ-140, DP-203
Final	Multi-cloud rollout + gamification	Cross-cloud labs, badges/points, optional Oracle / IBM / Alibaba

9.1 Build Steps per Phase

Detailed build steps, code and tests for every phase are in section 10 (Implementation Guide). Stage 1 = Phase 1, Stage 2 = Phase 2, Stage 3 = Phase 3, Stage 4 = Phase 4, Stage 5 = Final.

9.2 Testing per Phase

- Every phase repeats unit and integration testing for new features and re-runs regression.
- System, deployment and performance testing are run before each public release (APK/PWA).
- UAT is run with real learners at the end of Phase 1 and Phase 2 at minimum.

10. Implementation Guide - Stage by Stage

This section turns the roadmap into six build stages. Each stage lists a goal, the exact build order, the code, the tests that must pass and a Definition of Done. Finish one stage completely before starting the next.

10.1 How to Use This Guide

- **Work order inside every stage:** engine first (pure Python + tests), then API, then frontend, then content (theory, quiz, labs), then run the stage tests.
- **Write the test first** whenever a test ID is listed - the test IDs come from section 7.
- **Roadmap mapping:** Stage 1 = Phase 1, Stage 2 = Phase 2, Stage 3 = Phase 3, Stage 4 = Phase 4, Stage 5 = Final. Stage 0 is set-up.
- **Code files:** every file in this section is also in **cloudsim_starter.zip** (same folder layout), so you can copy the whole project instead of typing from the PDF.

What was verified. The engine and its tests were executed (40 tests, all passing) with Python 3.12. The FastAPI layer was wiring-checked with stand-in FastAPI/pydantic modules, not run under a real server. SQL, the React/Konva canvas, the service worker and the Pyodide loader are starter code: syntax-checked where possible, but not run in a browser or database. Run them once and adjust.

Stage	Roadmap phase	Main output	Tests to pass
Stage 0	Set-up	Repo, tools, database, dev servers	-
Stage 1	Phase 1 - AZ-900	CIDR/subnet engine, quiz, first API, canvas, first challenge lab	12 (test_stage1_network.py)
Stage 2	Phase 2 - AZ-104	Routing, firewall/NSG, IAM, VM lifecycle, storage, generic lab checker	16 (test_stage2_core.py + first 3 of test_stage2_labs.py)
Stage 3	Phase 3 - AWS + GCP	Provider adapters, cross-cloud peering, IAM conflicts, JSON/YAML export/import	9 (test_stage3_multicloud.py)
Stage 4	Phase 4 - Specialty	New-module recipe, new check types, AZ-140 lab	2 (last 2 of test_stage2_labs.py)
Stage 5	Final	Offline PWA/APK, on-device engine, gamification, class mode, release tests	1 + system/deployment tests

10.2 Stage 0 - Project Set-up

Goal

A repository where the backend tests run, the API starts, the database exists and the frontend dev server opens.

Tools

Tool	Version	Needed for
Python	3.11 or newer (engine tested on 3.12)	Engine, API, tests
Node.js	18 or newer	Frontend (Vite, React)
PostgreSQL	15 or newer	Database (from Stage 1)
Git + VS Code	any recent	Source control, editing
Android Studio	any recent	Stage 5 only, to build the APK

Folder layout

project tree

```
cloudsim/
  README.md
  backend/
    requirements.txt   pytest.ini
    app/
      main.py              FastAPI app (grows every stage)
      quiz.py              quiz scoring
      db/schema.sql        PostgreSQL tables
      engine/
        models.py  network.py  routing.py  firewall.py  iam.py
        compute.py  storage.py  validator.py  topology.py  gamification.py
        adapters/  azure.py  aws.py  gcp.py
    tests/             test_stage1 ... test_stage3, test_final
  labs/                lab definitions (JSON)
  content/             modules.json, <module>/lessons.json, quizzes/*.json
  frontend/
    index.html  package.json  vite.config.js
    public/            manifest.json  service-worker.js
    src/               main.jsx  Canvas.jsx  engine.js  progressStore.js  registerSW.js
```

Commands

terminal

```
git init cloudsim && cd cloudsim
python -m venv .venv && source .venv/bin/activate      # Windows: .venv\Scripts\activate
pip install -r backend/requirements.txt
createdb cloudsim && psql -d cloudsim -f backend/app/db/schema.sql

cd backend && pytest                                    # engine tests
uvicorn app.main:app --reload                           # API docs: http://127.0.0.1:8000/docs

cd ../frontend && npm install && npm run dev            # UI: http://localhost:5173
```

backend/requirements.txt

```
fastapi>=0.110
uvicorn[standard]>=0.29
PyYAML>=6.0
pytest>=8.0
httpx>=0.27
```

backend/pytest.ini

```
[pytest]
pythonpath = .
testpaths = tests
```

Frontend note: the starter uses Tailwind class names. Set up TailwindCSS once by following its Vite guide (tailwind.config.js content paths for index.html and src/**/*.{js,jsx}, plus the three @tailwind lines in a CSS file imported by main.jsx). vite.config.js already proxies /api to FastAPI on port 8000.

Definition of Done

- `pytest` runs (0 tests is fine at this point) and `uvicorn` starts.
- Database `cloudsim` exists.
- Vite dev server opens in the browser.

10.3 Stage 1 - Phase 1: AZ-900 Foundation

Goal

A learner can read AZ-900 lessons, take a quiz, build a VNet with subnets on the canvas, get instant CIDR validation and finish the first challenge lab (fix a subnet overlap).

Build order

1. **Database:** load schema.sql (Stage 0) and add the AZ-900 module row (SQL below). Lessons and quizzes start as JSON files in `content/`; move them into the lessons/quizzes tables later.
2. **Engine:** write `models.py` and `network.py` together with the Stage 1 tests. Tests first: UT-01, UT-02, UT-03, UT-04, NET-01.
3. **Quiz scoring:** `quiz.py` (UC-02, IT-10).
4. **Content files:** modules.json, az-900/lessons.json, quizzes/az900-q1.json.
5. **Lab file:** az900-lab02 (fix the subnet overlap).
6. **API:** main.py - CIDR check, module/lesson/quiz endpoints, lab list and lab validate.
7. **Frontend:** theory page (calls /api/modules and /api/modules/AZ-900/lessons), quiz page (POST /api/quizzes/az900-q1/submit) and the canvas (Canvas.jsx).
8. **Author the rest of the Stage 1 labs** (table below) and lessons for the three AZ-900 skill areas.
9. Run the tests and go through the Definition of Done.

Code

psql - add the AZ-900 module

```
INSERT INTO modules (certification_code, title, level, phase)
VALUES ('AZ-900', 'Microsoft Azure Fundamentals', 'fundamentals', 1);
```

backend/app/engine/models.py

```
from dataclasses import dataclass, field
from typing import List


@dataclass
class Subnet:
    name: str
    cidr: str


@dataclass
class Network:
    name: str
    provider: str            # "azure" | "aws" | "gcp"
    address_space: str       # e.g. "10.0.0.0/16"
    subnets: List[Subnet] = field(default_factory=list)
```

backend/app/engine/network.py

```
import ipaddress
from .models import Network, Subnet


class ValidationError(Exception):
    """Raised when a user action breaks a simulation rule."""


def parse_cidr(cidr):
    try:
        return ipaddress.ip_network(cidr, strict=True)
    except ValueError as e:
        raise ValidationError(f"Invalid CIDR '{cidr}': {e}")


def check_cidr_overlap(cidr1, cidr2):
    a, b = parse_cidr(cidr1), parse_cidr(cidr2)
    if a.version != b.version:
        return False
    return a.overlaps(b)
def subnet_within(network_cidr, subnet_cidr):
```

```
    net, sub = parse_cidr(network_cidr), parse_cidr(subnet_cidr)
    return net.version == sub.version and sub.subnet_of(net)


def add_subnet(network: Network, name: str, cidr: str) -> Subnet:
    """UT-01 / UT-04 / NET-01: validate, then add."""
    if any(s.name == name for s in network.subnets):
        raise ValidationError(f"Subnet name '{name}' already exists")
    if not subnet_within(network.address_space, cidr):
        raise ValidationError(
            f"Subnet {cidr} is not within address space {network.address_space}")
    for s in network.subnets:
        if check_cidr_overlap(s.cidr, cidr):
            raise ValidationError(f"Subnet {cidr} overlaps {s.name} ({s.cidr})")
    subnet = Subnet(name, cidr)
    network.subnets.append(subnet)
    return subnet


def check_peering(a: Network, b: Network):
    """IT-02: return a list of errors; empty list means peering is allowed."""
    if check_cidr_overlap(a.address_space, b.address_space):
        return [f"Address spaces overlap: {a.name} {a.address_space} "
                f"and {b.name} {b.address_space}"]
    return []


def find_issues(network: Network):
    """Used by challenge labs: list every problem in a network."""
    issues = []
    for i, s1 in enumerate(network.subnets):
        try:
            if not subnet_within(network.address_space, s1.cidr):
                issues.append(f"{s1.name} ({s1.cidr}) is outside {network.address_space}")
        except ValidationError as e:
            issues.append(str(e))
            continue
        for s2 in network.subnets[i + 1:]:
            try:
                if check_cidr_overlap(s1.cidr, s2.cidr):
                    issues.append(f"{s1.name} ({s1.cidr}) overlaps {s2.name} ({s2.cidr})")
            except ValidationError as e:
                issues.append(str(e))
    return issues
```

backend/app/quiz.py

```
"""Stage 1: score a quiz. Quiz JSON: {"id", "questions": [{"q", "options", "answer": index}]}"""


def score_quiz(quiz: dict, answers: list) -> dict:
    questions = quiz["questions"]
    if len(answers) != len(questions):
        raise ValueError("Number of answers does not match number of questions")
    correct = sum(1 for q, a in zip(questions, answers) if a == q["answer"])
    return {"correct": correct, "total": len(questions),
            "percent": round(100 * correct / len(questions)) if questions else 0}
```

content/modules.json

```
[
  {"code": "AZ-900", "title": "Microsoft Azure Fundamentals", "level": "fundamentals", "phase": 1}
]
```

content/az-900/lessons.json (shortened - full file in the ZIP)

```
[
  {"id": 1, "title": "What is cloud computing?",
   "content": "Cloud computing means renting computing resources over the internet ..."},
  {"id": 2, "title": "Regions and availability zones", "content": "..."},
  {"id": 3, "title": "Virtual networks and subnets", "content": "... Try it: open lab az900-lab02."}
]
```

content/quizzes/az900-q1.json

```
{
  "id": "az900-q1",
  "title": "Cloud concepts - quick check",
  "questions": [
    {"q": "Which service model gives you virtual machines where you manage the operating system?",
     "options": ["IaaS", "PaaS", "SaaS"], "answer": 0},
    {"q": "What is an Azure region?",
     "options": ["A set of datacenters within a latency-defined perimeter",
                 "A single physical server",
                 "A billing account"], "answer": 0},
    {"q": "In the shared responsibility model, who is always responsible for the physical datacenter?",
     "options": ["The customer", "The cloud provider", "A third-party auditor"], "answer": 1}
  ]
}
```

labs/az900_lab02_fix_subnet_overlap.json

```
{
  "id": "az900-lab02",
  "module": "AZ-900",
  "provider": "azure",
  "title": "Fix the subnet overlap",
  "difficulty": "beginner",
  "instructions": "Two subnets in this VNet overlap. Change one so the network has no issues.",
  "broken_state": {
    "name": "vnet-prod",
    "provider": "azure",
    "address_space": "10.0.0.0/16",
    "subnets": [
      {"name": "web", "cidr": "10.0.1.0/24"},
      {"name": "app", "cidr": "10.0.1.128/25"}
    ]
  },
  "goal_spec": {"max_issues": 0, "min_subnets": 2},
  "hints": [
    "Look at the CIDR ranges of the two subnets.",
    "10.0.1.128/25 lives inside 10.0.1.0/24.",
    "Move 'app' to a different /24, for example 10.0.2.0/24."
  ]
}
```

backend/app/main.py (Stage 1 version)

```
import json
import re
from pathlib import Path

from fastapi import FastAPI, HTTPException
from fastapi.responses import JSONResponse
from pydantic import BaseModel

from .engine import network
from .quiz import score_quiz
from .engine.models import Network, Subnet

app = FastAPI(title="CloudSim API")
ROOT = Path(__file__).resolve().parents[2]      # repo root
LABS_DIR = ROOT / "labs"
CONTENT_DIR = ROOT / "content"


@app.exception_handler(network.ValidationError)
def on_validation_error(request, exc):
    return JSONResponse(status_code=400, content={"detail": str(exc)})


class CidrPair(BaseModel):
    cidr1: str
    cidr2: str


class SubnetIn(BaseModel):
    name: str
    cidr: str


class NetworkIn(BaseModel):
    name: str
    provider: str
```

```
    address_space: str
    subnets: list[SubnetIn] = []


def to_network(n: NetworkIn) -> Network:
    return Network(n.name, n.provider, n.address_space,
                   [Subnet(s.name, s.cidr) for s in n.subnets])


def safe_slug(value: str) -> str:
    """Only letters, digits and dashes: blocks path tricks such as '../..'."""
    if not re.fullmatch(r"[A-Za-z0-9-]+", value):
        raise HTTPException(status_code=400, detail="Invalid name")
    return value.lower()


def read_json(path: Path):
    if not path.exists():
        raise HTTPException(status_code=404, detail="Not found")
    return json.loads(path.read_text())


def load_lab(lab_id: str) -> dict:
    for path in LABS_DIR.glob("*.json"):
        lab = json.loads(path.read_text())
        if lab["id"] == lab_id:
            return lab
    raise HTTPException(status_code=404, detail="Lab not found")


@app.post("/api/sim/network/check-cidr")
def check_cidr(p: CidrPair):
    return {"overlap": network.check_cidr_overlap(p.cidr1, p.cidr2)}


@app.get("/api/labs")
def list_labs():
    keys = ("id", "module", "provider", "title", "difficulty")
    labs = (json.loads(p.read_text()) for p in sorted(LABS_DIR.glob("*.json")))
    return [{k: lab[k] for k in keys} for lab in labs]


@app.post("/api/labs/{lab_id}/validate")
def validate_lab(lab_id: str, topology: NetworkIn):
    lab = load_lab(lab_id)
    net = to_network(topology)
    issues = network.find_issues(net)
    goal = lab["goal_spec"]
    passed = (len(issues) <= goal["max_issues"]
              and len(net.subnets) >= goal.get("min_subnets", 0))
    return {"passed": passed, "issues": issues}


@app.get("/api/modules")
def list_modules():
    return read_json(CONTENT_DIR / "modules.json")


@app.get("/api/modules/{code}/lessons")
def list_lessons(code: str):
    return read_json(CONTENT_DIR / safe_slug(code) / "lessons.json")


class QuizAnswers(BaseModel):
    answers: list[int]


@app.post("/api/quizzes/{quiz_id}/submit")
def submit_quiz(quiz_id: str, body: QuizAnswers):
    quiz = read_json(CONTENT_DIR / "quizzes" / f"{safe_slug(quiz_id)}.json")
    try:
        return score_quiz(quiz, body.answers)
    except ValueError as e:
        raise HTTPException(status_code=400, detail=str(e))
```

frontend/src/Canvas.jsx

```
import { useState } from "react";
import { Stage, Layer, Group, Rect, Text } from "react-konva";

const START = [
  { id: "vnet-prod", kind: "network", x: 30, y: 30, w: 440, h: 220, cidr: "10.0.0.0/16" },
  { id: "web", kind: "subnet", x: 50, y: 80, w: 180, h: 70, cidr: "10.0.1.0/24" },
  { id: "app", kind: "subnet", x: 260, y: 80, w: 180, h: 70, cidr: "10.0.1.128/25" },
];

export default function Canvas({ labId = "az900-lab02" }) {
  const [nodes, setNodes] = useState(START);
  const [result, setResult] = useState(null);

  const onDragEnd = (id, e) =>
    setNodes((ns) => ns.map((n) => (n.id === id ? { ...n, x: e.target.x(), y: e.target.y() } : n)));

  const editCidr = (id) => {
    const cidr = window.prompt("New CIDR for " + id);
    if (cidr) setNodes((ns) => ns.map((n) => (n.id === id ? { ...n, cidr } : n)));
  };

  async function validate() {
    const vnet = nodes.find((n) => n.kind === "network");
    const body = {
      name: vnet.id, provider: "azure", address_space: vnet.cidr,
      subnets: nodes.filter((n) => n.kind === "subnet").map((n) => ({ name: n.id, cidr: n.cidr })),
    };
    const res = await fetch(`/api/labs/${labId}/validate`, {
      method: "POST", headers: { "Content-Type": "application/json" }, body: JSON.stringify(body),
    });
    setResult(await res.json());
  }

  return (
    <div className="p-4">
      <button onClick={validate} className="px-4 py-2 bg-blue-700 text-white rounded">
        Validate
      </button>
      {result && (
        <p className={result.passed ? "text-green-700" : "text-red-700"}>
          {result.passed ? "Lab passed!" : result.issues.join(" | ")}
        </p>
      )}
      <Stage width={520} height={280} className="border mt-2">
        <Layer>
          {nodes.map((n) => (
            <Group key={n.id} x={n.x} y={n.y} draggable={n.kind !== "network"}
                   onDragEnd={(e) => onDragEnd(n.id, e)} onDblClick={() => editCidr(n.id)}>
              <Rect width={n.w} height={n.h} stroke="#12355B"
                    fill={n.kind === "network" ? "#EAF1F7" : "#CDE7EA"} />
              <Text text={`${n.id}\n${n.cidr}`} x={8} y={8} fontSize={13} />
            </Group>
          ))}
        </Layer>
      </Stage>
    </div>
  );
}
```

backend/tests/test_stage1_network.py

```
import json
from pathlib import Path
import pytest
from app.engine.models import Network, Subnet
from app.engine.network import (ValidationError, check_cidr_overlap, subnet_within,
                                add_subnet, find_issues)

LAB = Path(__file__).resolve().parents[2] / "labs" / "az900_lab02_fix_subnet_overlap.json"


def test_overlap_detected():                        # UT-01
    assert check_cidr_overlap("10.0.1.0/24", "10.0.1.0/25") is True


def test_no_overlap():                              # UT-02
    assert check_cidr_overlap("10.0.1.0/24", "10.0.2.0/24") is False
@pytest.mark.parametrize("bad", ["10.0.0.0/33", "300.1.1.1/24", "", "10.0.0.5/24"])
```

```
def test_invalid_cidr_rejected(bad):              # UT-03
    with pytest.raises(ValidationError):
        check_cidr_overlap(bad, "10.0.0.0/24")


def test_subnet_outside_network():                # UT-04
    assert subnet_within("10.0.0.0/16", "192.168.1.0/24") is False
    assert subnet_within("10.0.0.0/16", "10.0.5.0/24") is True


def test_add_subnet_overlap_error_message():      # NET-01
    net = Network("vnet1", "azure", "10.0.0.0/16")
    add_subnet(net, "web", "10.0.1.0/24")
    with pytest.raises(ValidationError) as e:
        add_subnet(net, "app", "10.0.1.0/25")
    assert "overlaps web" in str(e.value)


def test_add_subnet_outside_range():
    net = Network("vnet1", "azure", "10.0.0.0/16")
    with pytest.raises(ValidationError):
        add_subnet(net, "x", "192.168.0.0/24")


def test_challenge_lab_broken_then_fixed():       # UC-07 (challenge)
    lab = json.loads(LAB.read_text())
    s = lab["broken_state"]
    net = Network(s["name"], s["provider"], s["address_space"],
                  [Subnet(**x) for x in s["subnets"]])
    assert len(find_issues(net)) == 1             # broken: one overlap
    net.subnets[1].cidr = "10.0.2.0/24"           # the learner's fix
    assert len(find_issues(net)) <= lab["goal_spec"]["max_issues"]


def test_quiz_scoring():                          # UC-02 / IT-10
    from app.quiz import score_quiz
    quiz = {"id": "az900-q1", "questions": [
        {"q": "What does IaaS stand for?", "options": ["A", "B"], "answer": 0},
        {"q": "Who owns the physical datacenter in public cloud?", "options": ["A", "B"], "answer": 1}]}
    assert score_quiz(quiz, [0, 1]) == {"correct": 2, "total": 2, "percent": 100}
    assert score_quiz(quiz, [0, 0])["percent"] == 50
    with pytest.raises(ValueError):
        score_quiz(quiz, [0])


def test_content_files_are_valid():               # Stage 1 content
    from app.quiz import score_quiz
    root = Path(__file__).resolve().parents[2] / "content"
    quiz = json.loads((root / "quizzes" / "az900-q1.json").read_text())
    assert score_quiz(quiz, [0, 0, 1])["percent"] == 100
    lessons = json.loads((root / "az-900" / "lessons.json").read_text())
    assert [l["id"] for l in lessons] == [1, 2, 3]
    assert json.loads((root / "modules.json").read_text())[0]["code"] == "AZ-900"
```

Content to author in Stage 1

AZ-900 is organised into three skill areas: cloud concepts; Azure architecture and services; Azure management and governance. Check the current skills outline on Microsoft Learn before writing lessons, because exam outlines change.

Lab ID	Title	What the learner does	Engine functions
az900-lab01	Build your first VNet	Create VNet 10.0.0.0/16 with two non-overlapping subnets	add_subnet, find_issues
az900-lab02	Fix the subnet overlap (provided)	Change one of two overlapping subnets	find_issues
az900-lab03	Subnet outside the range	Correct a subnet that is not inside the VNet address space	subnet_within, find_issues

Tests

`cd backend && pytest tests/test_stage1_network.py` should report **12 passed**.

Definition of Done

- Test IDs UT-01 to UT-04, NET-01, UC-02 and the challenge test pass.
- POST /api/sim/network/check-cidr returns overlap true/false, and HTTP 400 with a clear message for an invalid CIDR.
- Lesson and quiz endpoints work; unsafe names such as ../.. are rejected (SP-02 style check).
- On the canvas: edit a subnet CIDR, press Validate, see the overlap message; after the fix the lab shows passed.
- Validation response time is measured locally and is well under 200 ms (baseline for SP-03).
- Hold a first UAT session (UAT-01, UAT-02) with 2-3 learners before starting Stage 2.

10.4 Stage 2 - Phase 2: AZ-104 Administrator

Goal

Administrator-level practice: routing, firewall/NSG rules, VNet peering, IAM, VM lifecycle and storage - plus a generic lab checker so new challenge labs are just JSON files.

Build order

1. **routing.py** - longest-prefix match, reachability, VNet peering (UT-05, UT-06, NET-03).
2. **firewall.py** - priority-ordered allow/deny evaluation (UT-13, NET-02).
3. **iam.py** - allow, explicit deny, implicit deny (UT-07, UT-08, UT-09, IAM-01, IAM-02).
4. **compute.py** - instance state machine with log entries (UT-10 to UT-12, CMP-01, CMP-02, CMP-04, CMP-05).
5. **storage.py** - quota and public-access guard (UT-14, STO-01, STO-03).
6. **validator.py** - the generic checker. Labs now use `goal_checks`; Stage 1 labs keep `goal_spec`.
7. **API** - route, firewall, IAM endpoints and `/api/labs/{id}/check`.
8. **Frontend** - add node types (VM, NSG, route table, storage, role assignment) with a property panel; Validate now posts the full state to /api/labs/{id}/check.
9. **Content** - AZ-104 lessons and labs (tables below).
10. Run tests, then UAT.

Code

backend/app/engine/routing.py

```
import ipaddress
from dataclasses import dataclass
from typing import List, Optional
from .network import ValidationError, parse_cidr, check_peering
from .models import Network


@dataclass
class Route:
    destination: str      # CIDR
    next_hop: str         # "local" | "internet" | "peering:<name>" | "appliance:<ip>" | "none"


def lookup(routes: List[Route], dest_ip: str) -> Optional[Route]:
    """UT-05 / UT-06: longest-prefix match. Returns None when nothing matches."""
    try:
        ip = ipaddress.ip_address(dest_ip)
    except ValueError:
        raise ValidationError(f"Invalid IP address '{dest_ip}'")
    best_net, best_route = None, None
    for r in routes:
        net = parse_cidr(r.destination)
        if net.version == ip.version and ip in net:
            if best_net is None or net.prefixlen > best_net.prefixlen:
                best_net, best_route = net, r
    return best_route


def can_reach(routes: List[Route], dest_ip: str) -> bool:
    r = lookup(routes, dest_ip)
    return r is not None and r.next_hop != "none"


def peer_networks(a: Network, b: Network, routes_a: List[Route], routes_b: List[Route]):
    """NET-03 / IT-01: refuse on overlap, otherwise add a route on BOTH sides."""
    errors = check_peering(a, b)
    if errors:
        return errors
    name = f"{a.name}-{b.name}"
    routes_a.append(Route(b.address_space, f"peering:{name}"))
    routes_b.append(Route(a.address_space, f"peering:{name}"))
    return []
```

backend/app/engine/firewall.py

```
import ipaddress
from dataclasses import dataclass
from typing import List, Optional, Tuple
from .network import ValidationError


@dataclass
class Rule:
    name: str
    priority: int                # lower number = evaluated first (Azure NSG / GCP style)
    direction: str               # "inbound" | "outbound"
    protocol: str                # "tcp" | "udp" | "icmp" | "*"
    port: Optional[int]          # None = any port
    source: str                  # CIDR or "*"
    action: str                  # "allow" | "deny"


def _matches(rule: Rule, direction, protocol, port, src_ip) -> bool:
    if rule.direction != direction:
        return False
    if rule.protocol not in ("*", protocol):
        return False
    if rule.port is not None and rule.port != port:
        return False
    if rule.source != "*":
        try:
            if ipaddress.ip_address(src_ip) not in ipaddress.ip_network(rule.source):
                return False
        except ValueError as e:
            raise ValidationError(str(e))
    return True


def evaluate(rules: List[Rule], direction, protocol, port, src_ip,
             default: str = "deny") -> Tuple[str, str]:
    """UT-13 / NET-02: first matching rule (by priority) wins.
    Returns (action, rule_name). Falls back to the provider default."""
    for rule in sorted(rules, key=lambda r: r.priority):
        if _matches(rule, direction, protocol, port, src_ip):
            return rule.action, rule.name
    return default, "default"
```

backend/app/engine/iam.py (Stage 2 part; find_conflicts is added in Stage 3)

```
from dataclasses import dataclass
from fnmatch import fnmatchcase
from typing import Dict, List


@dataclass
class Statement:
    effect: str                  # "allow" | "deny"
    actions: List[str]           # e.g. "storage:read", "storage:*", "*"
    resources: List[str]         # e.g. "bucket/logs", "bucket/*", "*"


@dataclass
class Decision:
    allowed: bool
    reason: str


def _any_match(patterns, value):
    return any(fnmatchcase(value, p) for p in patterns)


def evaluate(statements: List[Statement], action: str, resource: str) -> Decision:
    """UT-07/08/09: explicit deny beats allow; no match means implicit deny."""
    allowed = False
    for s in statements:
        if _any_match(s.actions, action) and _any_match(s.resources, resource):
            if s.effect == "deny":
                return Decision(False, "explicit deny")
            allowed = True
    return Decision(allowed, "allow" if allowed else "implicit deny")
```

backend/app/engine/compute.py

```
from dataclasses import dataclass, field
from datetime import datetime, timezone
from typing import List
from .models import Network
from .network import ValidationError

TRANSITIONS = {
    "creating": {"running"},
    "running": {"stopped", "deleted"},
    "stopped": {"running", "deleted"},
    "deleted": set(),
}


@dataclass
class Instance:
    name: str
    subnet: str
    state: str = "creating"
    log: List[str] = field(default_factory=list)

    def _log(self, message):
        self.log.append(f"{datetime.now(timezone.utc).isoformat()} {self.name}: {message}")

    def _move(self, new_state):
        if new_state not in TRANSITIONS[self.state]:
            raise ValidationError(f"Cannot go from '{self.state}' to '{new_state}'")
        self._log(f"{self.state} -> {new_state}")
        self.state = new_state

    def start(self):
        self._move("running")

    def stop(self):
        self._move("stopped")

    def delete(self):
        self._move("deleted")


def create_instance(network: Network, name: str, subnet_name: str) -> Instance:
    """UT-10 / UT-11 / CMP-01: needs a valid subnet; ends in state 'running'."""
    if not any(s.name == subnet_name for s in network.subnets):
        raise ValidationError(f"Subnet '{subnet_name}' not found in {network.name}")
    inst = Instance(name, subnet_name)
    inst._log("created")
    inst._move("running")
    return inst
```

backend/app/engine/storage.py

```
from dataclasses import dataclass, field
from typing import Dict
from .network import ValidationError


@dataclass
class Bucket:
    name: str
    quota_bytes: int
    public_access: bool = False          # bucket says "objects are public"
    block_public_access: bool = False    # account/bucket-level guard (S3 / GCP style)
    objects: Dict[str, int] = field(default_factory=dict)

    @property
    def used(self):
        return sum(self.objects.values())


def upload(bucket: Bucket, key: str, size: int):
    """UT-14 / STO-01: quota is enforced."""
    if size < 0:
        raise ValidationError("Object size cannot be negative")
    old = bucket.objects.get(key, 0)
    if bucket.used - old + size > bucket.quota_bytes:
        raise ValidationError(
            f"Quota exceeded: {bucket.used - old + size} > {bucket.quota_bytes} bytes")
    bucket.objects[key] = size
```

```
def anonymous_read_allowed(bucket: Bucket) -> bool:
    """STO-03: public access is blocked when the guard is on."""
    return bucket.public_access and not bucket.block_public_access
```

backend/app/engine/validator.py (Stage 2 version)

```
"""Stage 2: generic lab checker. A lab lists 'checks'; each check calls one engine function."""
from .models import Network, Subnet
from .network import find_issues
from .routing import Route, can_reach
from .firewall import Rule, evaluate as fw_eval
from .iam import Statement, evaluate as iam_eval


def state_from_dict(d: dict) -> dict:
    """Turn the JSON in a lab file (or a learner's canvas) into engine objects."""
    n = d["network"]
    return {
        "network": Network(n["name"], n["provider"], n["address_space"],
                           [Subnet(**s) for s in n.get("subnets", [])]),
        "routes": [Route(**r) for r in d.get("routes", [])],
        "rules": [Rule(**r) for r in d.get("rules", [])],
        "statements": [Statement(**s) for s in d.get("statements", [])],
    }


def run_checks(state: dict, checks: list) -> list:
    """Return a list of failure messages. Empty list = lab passed."""
    failures = []
    for c in checks:
        t = c["type"]
        if t == "no_subnet_issues":
            failures += find_issues(state["network"])
        elif t == "can_reach":
            if not can_reach(state["routes"], c["dest_ip"]):
                failures.append(f"No usable route to {c['dest_ip']}")
        elif t == "firewall_allows":
            action, rule = fw_eval(state["rules"], c["direction"], c["protocol"],
                                   c["port"], c["src_ip"])
            if action != "allow":
                failures.append(f"Port {c['port']} from {c['src_ip']} is blocked by '{rule}'")
        elif t in ("iam_allows", "iam_denies"):
            allowed = iam_eval(state["statements"], c["action"], c["resource"]).allowed
            if allowed != (t == "iam_allows"):
                failures.append(f"{c['action']} on {c['resource']} should be "
                                f"{'allowed' if t == 'iam_allows' else 'denied'}")
        else:
            raise ValueError(f"Unknown check type '{t}'")
    return failures
```

labs/az104_lab07_web_unreachable.json

```
{
  "id": "az104-lab07",
  "module": "AZ-104",
  "provider": "azure",
  "title": "Web server is unreachable",
  "difficulty": "intermediate",
  "instructions": "Internet users cannot open the website. Find and fix everything that blocks them.",
  "broken_state": {
    "network": {"name": "vnet-prod", "provider": "azure", "address_space": "10.0.0.0/16",
                "subnets": [{"name": "web", "cidr": "10.0.1.0/24"}]},
    "routes": [
      {"destination": "10.0.0.0/16", "next_hop": "local"},
      {"destination": "0.0.0.0/0", "next_hop": "none"}
    ],
    "rules": [
      {"name": "deny-all-inbound", "priority": 100, "direction": "inbound",
       "protocol": "*", "port": null, "source": "*", "action": "deny"}
    ],
    "statements": []
  },
  "goal_checks": [
    {"type": "can_reach", "dest_ip": "203.0.113.10"},
    {"type": "firewall_allows", "direction": "inbound", "protocol": "tcp",
     "port": 443, "src_ip": "203.0.113.10"}
  ],
  "hints": [
    "Traffic needs both a route and a firewall rule.",
    "Check where 0.0.0.0/0 points.",
```

```
    "Lower priority number wins: an allow rule for 443 must come before deny-all."
  ]
}
```

backend/app/main.py (add in Stage 2)

```
from .engine import routing, firewall, iam, compute, validator


class RouteIn(BaseModel):
    destination: str
    next_hop: str


class RuleIn(BaseModel):
    name: str
    priority: int
    direction: str
    protocol: str
    port: int | None = None
    source: str = "*"
    action: str


class StatementIn(BaseModel):
    effect: str
    actions: list[str]
    resources: list[str]


def to_routes(items): return [routing.Route(r.destination, r.next_hop) for r in items]
def to_rules(items): return [firewall.Rule(r.name, r.priority, r.direction, r.protocol,
                                           r.port, r.source, r.action) for r in items]
def to_statements(items): return [iam.Statement(s.effect, s.actions, s.resources) for s in items]


class RouteQuery(BaseModel):
    routes: list[RouteIn]
    dest_ip: str


@app.post("/api/sim/network/route")
def route(q: RouteQuery):
    found = routing.lookup(to_routes(q.routes), q.dest_ip)
    return {"route": None if found is None
            else {"destination": found.destination, "next_hop": found.next_hop}}


class FirewallQuery(BaseModel):
    rules: list[RuleIn]
    direction: str
    protocol: str
    port: int
    src_ip: str
    default: str = "deny"


@app.post("/api/sim/firewall/evaluate")
def firewall_evaluate(q: FirewallQuery):
    action, rule_name = firewall.evaluate(to_rules(q.rules), q.direction, q.protocol,
                                          q.port, q.src_ip, q.default)
    return {"action": action, "rule": rule_name}


class IamQuery(BaseModel):
    statements: list[StatementIn]
    action: str
    resource: str


@app.post("/api/sim/iam/evaluate")
def iam_evaluate(q: IamQuery):
    d = iam.evaluate(to_statements(q.statements), q.action, q.resource)
    return {"allowed": d.allowed, "reason": d.reason}


class InstanceIn(BaseModel):
    name: str
    subnet: str
    state: str = "running"
```

```
class LabState(BaseModel):
    network: NetworkIn
    routes: list[RouteIn] = []
    rules: list[RuleIn] = []
    statements: list[StatementIn] = []
    instances: list[InstanceIn] = []


@app.post("/api/labs/{lab_id}/check")          # used by labs that define "goal_checks"
def check_lab(lab_id: str, s: LabState):
    lab = load_lab(lab_id)
    state = {"network": to_network(s.network), "routes": to_routes(s.routes),
             "rules": to_rules(s.rules), "statements": to_statements(s.statements),
             "instances": [compute.Instance(i.name, i.subnet, i.state) for i in s.instances]}
    failures = validator.run_checks(state, lab["goal_checks"])
    return {"passed": not failures, "issues": failures}
```

backend/tests/test_stage2_core.py

```
import pytest
from app.engine.models import Network
from app.engine.network import ValidationError, add_subnet
from app.engine.routing import Route, lookup, can_reach
from app.engine.firewall import Rule, evaluate
from app.engine.iam import Statement, evaluate as iam_eval
from app.engine.compute import create_instance
from app.engine.storage import Bucket, upload, anonymous_read_allowed


def test_longest_prefix_match():                    # UT-05
    routes = [Route("10.0.0.0/16", "local"), Route("10.0.1.0/24", "appliance:10.0.9.4")]
    assert lookup(routes, "10.0.1.5").next_hop == "appliance:10.0.9.4"
    assert lookup(routes, "10.0.2.5").next_hop == "local"


def test_no_matching_route():                       # UT-06
    assert lookup([Route("10.0.0.0/16", "local")], "8.8.8.8") is None
    assert can_reach([Route("10.0.0.0/16", "local")], "8.8.8.8") is False


def test_firewall_priority_and_default():           # UT-13 / NET-02
    rules = [Rule("allow-ssh", 200, "inbound", "tcp", 22, "*", "allow"),
             Rule("deny-ssh", 100, "inbound", "tcp", 22, "*", "deny")]
    assert evaluate(rules, "inbound", "tcp", 22, "1.2.3.4") == ("deny", "deny-ssh")
    assert evaluate(rules, "inbound", "tcp", 443, "1.2.3.4") == ("deny", "default")


def test_firewall_source_cidr():
    rules = [Rule("office", 100, "inbound", "tcp", 443, "203.0.113.0/24", "allow")]
    assert evaluate(rules, "inbound", "tcp", 443, "203.0.113.7")[0] == "allow"
    assert evaluate(rules, "inbound", "tcp", 443, "198.51.100.7")[0] == "deny"


def test_iam_allow_deny_implicit():                 # UT-07 / UT-08 / UT-09
    allow = Statement("allow", ["storage:read"], ["bucket/*"])
    deny = Statement("deny", ["storage:read"], ["bucket/secret"])
    assert iam_eval([allow], "storage:read", "bucket/logs").allowed is True
    d = iam_eval([allow, deny], "storage:read", "bucket/secret")
    assert d.allowed is False and d.reason == "explicit deny"
    d = iam_eval([], "storage:read", "bucket/logs")
    assert d.allowed is False and d.reason == "implicit deny"


def test_instance_lifecycle_and_log():              # UT-10 / UT-12 / CMP-01 / CMP-02
    net = Network("vnet", "azure", "10.0.0.0/16")
    add_subnet(net, "web", "10.0.1.0/24")
    vm = create_instance(net, "vm1", "web")
    assert vm.state == "running"
    vm.stop(); vm.start()
    assert vm.state == "running" and len(vm.log) == 4


def test_instance_needs_subnet():                   # UT-11
    with pytest.raises(ValidationError):
        create_instance(Network("vnet", "azure", "10.0.0.0/16"), "vm1", "missing")


def test_stop_twice_is_error():                     # CMP-04
    net = Network("vnet", "aws", "10.0.0.0/16")
    add_subnet(net, "a", "10.0.1.0/24")
```

```
    vm = create_instance(net, "i-1", "a")
    vm.stop()
    with pytest.raises(ValidationError):
        vm.stop()


def test_storage_quota():                          # UT-14 / STO-01
    b = Bucket("logs", quota_bytes=100)
    upload(b, "a.txt", 60)
    with pytest.raises(ValidationError):
        upload(b, "b.txt", 60)


def test_public_access_blocked():                  # STO-03
    b = Bucket("site", 1000, public_access=True, block_public_access=True)
    assert anonymous_read_allowed(b) is False


def test_bad_ip_is_validation_error():
    with pytest.raises(ValidationError):
        lookup([Route("10.0.0.0/16", "local")], "not-an-ip")
    rules = [Rule("r", 1, "inbound", "tcp", 22, "10.0.0.0/8", "allow")]
    with pytest.raises(ValidationError):
        evaluate(rules, "inbound", "tcp", 22, "bad")


def test_azure_vnet_peering_updates_routes():      # NET-03
    from app.engine.routing import peer_networks
    a = Network("vnet-a", "azure", "10.0.0.0/16")
    b = Network("vnet-b", "azure", "10.1.0.0/16")
    ra, rb = [Route("10.0.0.0/16", "local")], [Route("10.1.0.0/16", "local")]
    assert peer_networks(a, b, ra, rb) == []
    assert ra[-1] == Route("10.1.0.0/16", "peering:vnet-a-vnet-b")
    assert rb[-1] == Route("10.0.0.0/16", "peering:vnet-a-vnet-b")


def test_delete_running_instance():                # CMP-05
    net = Network("vnet", "azure", "10.0.0.0/16")
    add_subnet(net, "web", "10.0.1.0/24")
    vm = create_instance(net, "vm1", "web")
    vm.delete()
    assert vm.state == "deleted" and "running -> deleted" in vm.log[-1]
    with pytest.raises(ValidationError):
        vm.start()
```

backend/tests/test_stage2_labs.py (Stage 2 part)

```
import json
from pathlib import Path
import pytest
from app.engine.validator import state_from_dict, run_checks
from app.engine.routing import Route
from app.engine.firewall import Rule

LAB = Path(__file__).resolve().parents[2] / "labs" / "az104_lab07_web_unreachable.json"


def test_broken_lab_then_fixed():                  # UC-07 (routing + firewall challenge)
    lab = json.loads(LAB.read_text())
    state = state_from_dict(lab["broken_state"])
    assert len(run_checks(state, lab["goal_checks"])) == 2       # both problems found

    state["routes"][1] = Route("0.0.0.0/0", "internet")           # fix 1: default route
    state["rules"].append(Rule("allow-https", 90, "inbound", "tcp", 443, "*", "allow"))  # fix 2
    assert run_checks(state, lab["goal_checks"]) == []


def test_iam_checks():                             # IAM-04 (least privilege)
    state = state_from_dict({
        "network": {"name": "n", "provider": "azure", "address_space": "10.0.0.0/16"},
        "statements": [{"effect": "allow", "actions": ["*"], "resources": ["*"]}]})
    checks = [{"type": "iam_denies", "action": "storage:delete", "resource": "bucket/logs"}]
    assert len(run_checks(state, checks)) == 1                    # over-permissive: fails
    state["statements"][0].actions = ["storage:read"]             # learner applies least privilege
    assert run_checks(state, checks) == []


def test_unknown_check_type():
    state = state_from_dict({"network": {"name": "n", "provider": "azure",
                                         "address_space": "10.0.0.0/16"}})
```

```
    with pytest.raises(ValueError):
        run_checks(state, [{"type": "nope"}])
```

AZ-104 content plan

AZ-104 covers five areas: identities and governance; storage; compute; virtual networking; monitoring and maintenance. Confirm the current outline on Microsoft Learn, then write one lesson group and one quiz per area.

Lab ID	Title	Checks / engine used	Test IDs
az104-lab01	Build a VNet with three subnets	add_subnet, find_issues	UT-01 to UT-04
az104-lab02	Allow HTTPS, block SSH	firewall_allows + evaluate	NET-02, UT-13
az104-lab03	Peer two VNets	peer_networks, can_reach	NET-03
az104-lab04	VM lifecycle	create_instance, start/stop/delete	CMP-01, CMP-02, CMP-04, CMP-05
az104-lab05	Storage quota and public access	upload, anonymous_read_allowed	STO-01, STO-03
az104-lab06	Least-privilege role	iam_allows / iam_denies checks	IAM-01, IAM-02, IAM-04
az104-lab07	Web server is unreachable (provided)	can_reach + firewall_allows	UC-07 challenge

Tests

`pytest tests/test_stage2_core.py tests/test_stage2_labs.py` - in this stage the first 3 tests of test_stage2_labs.py apply. Expected: **16 passed** for Stage 2 (the two remaining tests in test_stage2_labs.py belong to Stage 4 and need the Stage 4 change).

Still for you to write: STO-04 (Blob anonymous access vs IAM deny), STO-05 (delete object with a read-only role) and IAM-05 (role inheritance by scope). They combine storage with IAM, so decide the precedence rules first and document them in the lesson.

Definition of Done

- All Stage 2 tests pass; longest-prefix, priority order and explicit-deny-wins are covered.
- az104-lab07 goes from failing (2 problems reported) to passing after the learner fixes the route and adds the allow rule.
- Each new lab is only a JSON file plus content - no engine change needed.
- Validation stays under 200 ms with a lab of about 20 resources (SP-03, SP-06).
- UAT-01 to UAT-04 run with real learners; feedback messages judged clear.

10.5 Stage 3 - Phase 3: AWS + GCP Integration

Goal

The same engine speaks three vocabularies (Azure, AWS, GCP), two clouds can be connected safely, IAM conflicts across clouds are reported, and a lab can be exported and imported as JSON or YAML.

Build order

1. **Adapters** - one small file per provider with its names and rule quirks (AWS Security Groups are allow-only). Wire `validate_rule` into the place where a firewall rule is added.
2. **Provider vocabulary in the UI** - copy each adapter's NAMES into `frontend/src/providerNames.js` and show provider words on the canvas.
3. **Cross-cloud connectivity** - reuse `peer_networks` from Stage 2 (IT-01, IT-02, IT-03).
4. **IAM conflicts** - add `find_conflicts` to iam.py (IAM-03).
5. **Export/import** - topology.py with JSON and YAML (IT-06, IT-07, IT-08, SP-02).
6. **API** - export and import endpoints.
7. **Content and labs** - AWS and GCP lessons and labs (table below).
8. Run tests, then integration and UAT.

Provider differences the labs must teach

Topic	Azure	AWS	GCP
Network	VNet (regional)	VPC (regional); a subnet lives in one Availability Zone	VPC network (global); subnets are regional
Firewall	NSG: allow and deny rules, priority number, lower wins	Security Group: allow rules only, no priority. Network ACL: numbered allow and deny, stateless	VPC firewall rules: allow and deny, priority number, lower wins
Default inbound	Deny except VNet and load balancer traffic	Security Group denies all inbound	Implied deny for ingress
Compute	Virtual Machine	EC2 instance	Compute Engine instance
Storage	Storage account + Blob container	S3 bucket	Cloud Storage bucket
IAM	RBAC role assignments at a scope	IAM policies (JSON statements); explicit deny wins	IAM roles bound to a resource hierarchy level

Version 1 of the simulator does not model regions or availability zones: every network is one address space with subnets. Add zones later as a separate feature.

Code

backend/app/engine/adapters/__init__.py

```
"""Stage 3: provider adapters. Each maps provider words to the common model."""
from . import azure, aws, gcp

ADAPTERS = {"azure": azure, "aws": aws, "gcp": gcp}
```

backend/app/engine/adapters/azure.py

```
NAMES = {"network": "Virtual Network (VNet)", "firewall": "Network Security Group (NSG)",
         "compute": "Virtual Machine", "storage": "Blob container", "iam": "Azure RBAC"}
FIREWALL_USES_PRIORITY = True


def validate_rule(rule):
    """NSG rules can allow or deny and always have a priority."""
    return None
```

backend/app/engine/adapters/aws.py

```
from ..network import ValidationError

NAMES = {"network": "Virtual Private Cloud (VPC)", "firewall": "Security Group",
         "compute": "EC2 instance", "storage": "S3 bucket", "iam": "AWS IAM"}
FIREWALL_USES_PRIORITY = False   # Security Groups have no priority


def validate_rule(rule):
    """Security Groups are allow-only: a deny rule is a learner mistake."""
    if rule.action == "deny":
        raise ValidationError("AWS Security Groups only support allow rules. "
                              "Use a Network ACL if you need to deny traffic.")
```

backend/app/engine/adapters/gcp.py

```
NAMES = {"network": "VPC network", "firewall": "Firewall rule",
         "compute": "Compute Engine instance", "storage": "Cloud Storage bucket", "iam": "GCP IAM"}
FIREWALL_USES_PRIORITY = True


def validate_rule(rule):
    """VPC firewall rules can allow or deny and have a priority."""
    return None
```

backend/app/engine/iam.py (add to the end of the file)

```
def find_conflicts(policies: Dict[str, List[Statement]]):
    """IAM-03: the same action+resource allowed in one cloud but denied in another."""
    conflicts = []
    clouds = list(policies)
    for i, c1 in enumerate(clouds):
        for c2 in clouds[i + 1:]:
            for s1 in policies[c1]:
                for s2 in policies[c2]:
                    if s1.effect == s2.effect:
                        continue
                    shared_actions = set(s1.actions) & set(s2.actions)
                    shared_res = set(s1.resources) & set(s2.resources)
                    for a in sorted(shared_actions):
                        for r in sorted(shared_res):
                            allow_cloud = c1 if s1.effect == "allow" else c2
                            deny_cloud = c2 if s1.effect == "allow" else c1
                            conflicts.append(
                                f"{a} on {r}: allowed in {allow_cloud}, denied in {deny_cloud}")
    return conflicts
```

backend/app/engine/topology.py

```
"""Stage 3: export / import a multi-cloud lab as JSON or YAML (IT-06, IT-07, IT-08)."""
import json
from dataclasses import asdict
from typing import List
import yaml
from .models import Network, Subnet


def to_dict(networks: List[Network]) -> dict:
    return {"version": 1, "networks": [asdict(n) for n in networks]}


def from_dict(data: dict) -> List[Network]:
    if not isinstance(data, dict) or data.get("version") != 1 or "networks" not in data:
        raise ValueError("Unsupported or malformed lab file")
    return [Network(n["name"], n["provider"], n["address_space"],
                    [Subnet(**s) for s in n.get("subnets", [])])
            for n in data["networks"]]


def export_lab(networks: List[Network], fmt: str = "json") -> str:
    data = to_dict(networks)
    if fmt == "json":
        return json.dumps(data, indent=2)
    if fmt == "yaml":
        return yaml.safe_dump(data, sort_keys=False)
    raise ValueError("fmt must be 'json' or 'yaml'")


def import_lab(text: str, fmt: str = "json") -> List[Network]:
    try:
```

```
        data = json.loads(text) if fmt == "json" else yaml.safe_load(text)
    except (json.JSONDecodeError, yaml.YAMLError) as e:
        raise ValueError(f"Cannot parse {fmt}: {e}")
    return from_dict(data)
```

backend/app/main.py (add in Stage 3)

```
from fastapi import Response
from .engine import topology


class LabIn(BaseModel):
    networks: list[NetworkIn]


class ImportIn(BaseModel):
    text: str
    format: str = "json"


@app.post("/api/lab-configs/export")
def export_lab(lab: LabIn, format: str = "json"):
    text = topology.export_lab([to_network(n) for n in lab.networks], format)
    media = "application/json" if format == "json" else "application/x-yaml"
    return Response(content=text, media_type=media)


@app.post("/api/lab-configs/import")
def import_lab(body: ImportIn):
    try:
        nets = topology.import_lab(body.text, body.format)
    except (ValueError, KeyError, TypeError) as e:
        raise HTTPException(status_code=400, detail=f"Bad lab file: {e}")
    return topology.to_dict(nets)
```

backend/tests/test_stage3_multicloud.py

```
import json
import yaml
import pytest
from app.engine.models import Network
from app.engine.network import add_subnet
from app.engine.routing import peer_networks, can_reach
from app.engine.iam import Statement, find_conflicts
from app.engine.topology import export_lab, import_lab
from app.engine.adapters import ADAPTERS


def _pair(cidr_b="10.1.0.0/16"):
    return (Network("azure-vnet", "azure", "10.0.0.0/16"),
            Network("aws-vpc", "aws", cidr_b))


def test_azure_aws_connectivity():                # IT-01 / NET-03
    a, b = _pair()
    ra, rb = [], []
    assert peer_networks(a, b, ra, rb) == []
    assert can_reach(ra, "10.1.4.4") and can_reach(rb, "10.0.4.4")


def test_overlapping_cidr_refused():              # IT-02
    a, b = _pair(cidr_b="10.0.0.0/16")
    ra, rb = [], []
    errors = peer_networks(a, b, ra, rb)
    assert errors and "overlap" in errors[0] and ra == [] and rb == []


def test_missing_return_route():                  # IT-03
    a, b = _pair()
    ra, rb = [], []
    peer_networks(a, b, ra, rb)
    rb.clear()
    assert can_reach(ra, "10.1.4.4") is True
    assert can_reach(rb, "10.0.4.4") is False     # reply cannot return


def test_multicloud_role_conflict():              # IAM-03
    policies = {
        "azure": [Statement("allow", ["storage:read"], ["data/reports"])],
        "aws": [Statement("deny", ["storage:read"], ["data/reports"])],
```

```
    }
    conflicts = find_conflicts(policies)
    assert conflicts == ["storage:read on data/reports: allowed in azure, denied in aws"]


def test_export_import_roundtrip_json_and_yaml():  # IT-06 / IT-07 / IT-08
    a, b = _pair()
    add_subnet(a, "web", "10.0.1.0/24")
    add_subnet(b, "app", "10.1.1.0/24")
    for fmt in ("json", "yaml"):
        text = export_lab([a, b], fmt)
        assert import_lab(text, fmt) == [a, b]
    assert json.loads(export_lab([a, b], "json")) == yaml.safe_load(export_lab([a, b], "yaml"))


def test_import_rejects_bad_file():               # SP-02
    with pytest.raises(ValueError):
        import_lab('{"version": 99}', "json")


def test_adapters_use_provider_words():
    assert "VPC" in ADAPTERS["aws"].NAMES["network"]
    assert "VNet" in ADAPTERS["azure"].NAMES["network"]
    assert ADAPTERS["aws"].FIREWALL_USES_PRIORITY is False


def test_import_rejects_bad_yaml_and_wrong_shape():
    with pytest.raises(ValueError):
        import_lab("a: [unclosed", "yaml")
    with pytest.raises(ValueError):
        import_lab("just a string", "yaml")


def test_aws_security_group_rejects_deny_rule():
    from app.engine.firewall import Rule
    from app.engine.network import ValidationError
    deny = Rule("d", 1, "inbound", "tcp", 22, "*", "deny")
    allow = Rule("a", 1, "inbound", "tcp", 22, "*", "allow")
    with pytest.raises(ValidationError):
        ADAPTERS["aws"].validate_rule(deny)
    assert ADAPTERS["aws"].validate_rule(allow) is None
    assert ADAPTERS["azure"].validate_rule(deny) is None
    assert ADAPTERS["gcp"].validate_rule(deny) is None
```

Labs to author in Stage 3

Lab ID	Provider	Task	Test IDs
aws-lab01	AWS	Build a VPC with public and private subnets	UT-01 to UT-04
aws-lab02	AWS	Start and stop an EC2 instance; read the log	CMP-02
aws-lab03	AWS	S3 bucket: IAM deny blocks a user	STO-02
aws-lab04	AWS	Security Group: replace a deny rule with allow-only design	aws validate_rule
gcp-lab01	GCP	VPC + firewall rule applied to a Compute Engine instance	CMP-03
gcp-lab02	GCP	Block public access on a bucket	STO-03
multi-lab01	Azure + AWS	Connect a VNet and a VPC without overlap	IT-01, IT-02, IT-03
multi-lab02	Azure + AWS	Resolve a role conflict between the two clouds	IAM-03
multi-lab03	Any	Export a lab as YAML, edit it, import it back	IT-06 to IT-08

Tests

`pytest tests/test_stage3_multicloud.py` should report **9 passed**.

Definition of Done

- Overlapping address spaces are refused; non-overlapping ones get routes on both sides; a missing return route is detected.
- The same lab can be opened with Azure, AWS or GCP words and behaves the same (ST-07).
- Export then import gives an identical topology in JSON and in YAML.

- Malformed, wrong-version or oversized lab files are rejected cleanly, never crash the server (SP-02).
- AWS deny rules on Security Groups are rejected with the helpful message.

10.6 Stage 4 - Phase 4: Specialty Modules

Goal

Add AZ-305, AZ-400, AZ-140 and DP-203 without changing the engine core. From this stage on, a new module is a repeatable recipe.

New-module recipe

1. Add a row to the modules table and to `content/modules.json`.
2. Write `content/<code>/lessons.json` and a quiz in `content/quizzes/`.
3. Design labs using the existing check types: no_subnet_issues, can_reach, firewall_allows, iam_allows, iam_denies.
4. If a lab needs something new, add **one** check type to validator.py and a unit test for it.
5. Add a canvas node type only if the lab needs a new kind of object.
6. Add the new test IDs to the traceability table and run the full suite.

Microsoft retires and renames certifications from time to time. Check the current exam list on Microsoft Learn before authoring content for each module.

Modules in this stage

Module	What the learner practises	Existing checks reused	New check / node
AZ-305 Solutions Architect	Design review: arrange a topology that meets written requirements	no_subnet_issues, can_reach, firewall_allows, iam_denies	no_public_storage (uses anonymous_read_allowed)
AZ-400 DevOps Engineer	Infrastructure as code: export a lab as YAML, repair a broken file, order pipeline stages	topology export/import	stage_order check + pipeline node
AZ-140 Azure Virtual Desktop	Host pool of session hosts; bring the pool online	no_subnet_issues, iam_allows	min_running_instances (code below)
DP-203 Data Engineer	Data lake storage with quota; ingest, transform, serve flow	storage quota, iam checks	data_flow_connected check + flow node

The remaining certifications in the theory list (AZ-120, DP-420, and the Associate and Fundamentals modules AI-901, DP-900, SC-900, AZ-204, AI-200, AZ-500, SC-500, DP-300, PL-300, AZ-700, SC-100) follow the same recipe. Start each one as theory + quiz only, and add labs when a matching check exists.

Code - the first new check (AZ-140)

backend/app/engine/validator.py (Stage 4 changes)

```
# 1) top of file: add the import
from .compute import Instance

# 2) state_from_dict(): add one line to the returned dictionary
        "instances": [Instance(**i) for i in d.get("instances", [])],

# 3) run_checks(): add this branch before the final 'else'
        elif t == "min_running_instances":
            running = sum(1 for i in state.get("instances", []) if i.state == "running")
            if running < c["count"]:
                failures.append(f"Only {running} of {c['count']} required instances are running")
```

The API needs no change: LabState already accepts instances (Stage 2).

labs/az140_lab01_host_pool.json

```
{
  "id": "az140-lab01",
  "module": "AZ-140",
  "provider": "azure",
  "title": "Bring the host pool online",
  "difficulty": "advanced",
  "instructions": "The desktop host pool needs 3 running session hosts. Start the missing ones.",
  "broken_state": {
    "network": {"name": "vnet-avd", "provider": "azure", "address_space": "10.10.0.0/16",
                "subnets": [{"name": "hosts", "cidr": "10.10.1.0/24"}]},
    "instances": [
      {"name": "host1", "subnet": "hosts", "state": "running"},
      {"name": "host2", "subnet": "hosts", "state": "stopped"},
      {"name": "host3", "subnet": "hosts", "state": "stopped"}
    ]
  },
  "goal_checks": [
    {"type": "no_subnet_issues"},
    {"type": "min_running_instances", "count": 3}
  ]
}
```

backend/tests/test_stage2_labs.py (add in Stage 4)

```
def test_min_running_instances_check():            # Stage 4 (AZ-140 style host pool)
    state = state_from_dict({
        "network": {"name": "n", "provider": "azure", "address_space": "10.0.0.0/16"},
        "instances": [{"name": "host1", "subnet": "s", "state": "running"},
                      {"name": "host2", "subnet": "s", "state": "stopped"}]})
    checks = [{"type": "min_running_instances", "count": 2}]
    assert run_checks(state, checks) == ["Only 1 of 2 required instances are running"]
    state["instances"][1].state = "running"
    assert run_checks(state, checks) == []


def test_host_pool_lab():                          # Stage 4 lab file
    lab = json.loads((LAB.parent / "az140_lab01_host_pool.json").read_text())
    state = state_from_dict(lab["broken_state"])
    assert len(run_checks(state, lab["goal_checks"])) == 1        # only 1 of 3 running
    for inst in state["instances"]:
        if inst.state == "stopped":
            inst.start()                                          # the learner's fix
    assert run_checks(state, lab["goal_checks"]) == []
```

Tests

`pytest` now runs the whole suite: **40 passed** (Stage 4 adds 2 tests).

Definition of Done

- Each of the four modules has lessons, a quiz and at least one lab.
- Every new check type has a unit test and a clear failure message.
- Full regression suite still passes (no engine core change was needed).
- UAT with learners for at least one specialty module.

10.7 Stage 5 - Final: Offline Release + Gamification

Goal

Everything works offline on a phone or browser, progress lives on the device, learners earn badges, trainers can collect class results without a server, and the APK and PWA are release-tested.

Build order

1. **Run the engine on the device.** Recommended: Pyodide (Python compiled to WebAssembly) so the same engine.py files run in the browser. Alternative: rewrite the engine in TypeScript (more work, smaller download).
2. **Progress store** on the device (localStorage for small data; move to IndexedDB if it grows).
3. **Gamification** - badge rules in Python (or port to JS) and a badge screen.
4. **PWA files** - manifest, service worker, registration.
5. **Class mode without a server** - students export their progress file, the trainer imports many files into one table (UC-12).
6. **APK** - wrap the PWA build with Capacitor.
7. **Release testing** - system, security/performance and deployment test cases.

Code

frontend/src/engine.js

```
// Stage 5: run the SAME Python engine inside the browser/APK with Pyodide (no server needed).
import { loadPyodide } from "pyodide";

const ENGINE_FILES = ["__init__", "models", "network", "routing", "firewall", "iam",
                      "compute", "storage", "validator", "topology"];
let py;

export async function initEngine() {
  py = await loadPyodide({ indexURL: "/pyodide/" });   // self-host so it works offline
  await py.loadPackage("pyyaml");
  py.FS.mkdirTree("/home/pyodide/app/engine");
  py.FS.writeFile("/home/pyodide/app/__init__.py", "");
  for (const f of ENGINE_FILES) {
    const src = await (await fetch(`/engine/${f}.py`)).text();   // copy engine/*.py into public/engine/
    py.FS.writeFile(`/home/pyodide/app/engine/${f}.py`, src);
  }
  py.runPython("import sys; sys.path.insert(0, '/home/pyodide')");
}

export function checkCidr(a, b) {
  py.globals.set("cidr_a", a);
  py.globals.set("cidr_b", b);
  return py.runPython(
    "from app.engine.network import check_cidr_overlap\ncheck_cidr_overlap(cidr_a, cidr_b)");
}
```

Copy the engine files and the Pyodide runtime into the frontend so nothing is downloaded at run time: `mkdir -p frontend/public/engine frontend/public/pyodide`, then copy backend/app/engine/*.py into public/engine and the files from node_modules/pyodide into public/pyodide (needs `npm install pyodide`). The service worker then caches them for offline use. Pyodide adds a multi-megabyte first download and is slower on low-end phones, so measure it in DT-03.

frontend/src/progressStore.js

```
// Stage 5: progress is stored on the device only (no server, no billing).
const KEY = "cloudsim-progress-v1";

export const loadProgress = () => JSON.parse(localStorage.getItem(KEY) || "{}");
export const saveProgress = (p) => localStorage.setItem(KEY, JSON.stringify(p));

export function completeLab(labId, provider) {
  const p = loadProgress();
  p.labs = { ...(p.labs || {}), [labId]: { completedAt: new Date().toISOString(), provider } };
  saveProgress(p);
  return p;
}
```

```
// Trainer / class mode: students export this file, trainer imports many of them.
export const exportProgressFile = () =>
  new Blob([JSON.stringify(loadProgress(), null, 2)], { type: "application/json" });
```

backend/app/engine/gamification.py

```
"""Final stage: simple badge rules based on progress numbers."""

BADGES = [
    ("First Lab", lambda p: p.get("labs_completed", 0) >= 1),
    ("Network Builder", lambda p: p.get("network_labs", 0) >= 3),
    ("Bug Hunter", lambda p: p.get("challenges_completed", 0) >= 3),
    ("Multi-Cloud Explorer", lambda p: p.get("providers_used", 0) >= 3),
    ("Quiz Master", lambda p: p.get("best_quiz_score", 0) >= 90),
]


def award_badges(progress: dict):
    return [name for name, rule in BADGES if rule(progress)]
```

backend/tests/test_final_gamification.py

```
from app.engine.gamification import award_badges


def test_badges():
    assert award_badges({}) == []
    got = award_badges({"labs_completed": 1, "providers_used": 3, "best_quiz_score": 95})
    assert got == ["First Lab", "Multi-Cloud Explorer", "Quiz Master"]
```

frontend/public/manifest.json

```
{
  "name": "Multi-Cloud Simulator",
  "short_name": "CloudSim",
  "start_url": "/",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#12355B",
  "icons": [
    { "src": "/icon-192.png", "sizes": "192x192", "type": "image/png" },
    { "src": "/icon-512.png", "sizes": "512x512", "type": "image/png" }
  ]
}
```

frontend/public/service-worker.js

```
// Stage 5: cache-first service worker so the app works offline.
const CACHE = "cloudsim-v1";
const APP_SHELL = ["/", "/index.html", "/manifest.json"];

self.addEventListener("install", (e) => {
  e.waitUntil(caches.open(CACHE).then((c) => c.addAll(APP_SHELL)));
  self.skipWaiting();
});

self.addEventListener("activate", (e) => {
  e.waitUntil(
    caches.keys().then((keys) =>
      Promise.all(keys.filter((k) => k !== CACHE).map((k) => caches.delete(k))))
  );
});

self.addEventListener("fetch", (e) => {
  if (e.request.method !== "GET") return;
  e.respondWith(
    caches.match(e.request).then((hit) =>
      hit || fetch(e.request).then((res) => {
        const copy = res.clone();
        caches.open(CACHE).then((c) => c.put(e.request, copy));
        return res;
      })
    )
  );
});
```

frontend/src/registerSW.js

```
export function registerServiceWorker() {
  if ("serviceWorker" in navigator) {
    window.addEventListener("load", () => navigator.serviceWorker.register("/service-worker.js"));
  }
}
```

terminal - build the Android APK with Capacitor

```
cd frontend
npm run build                                  # creates dist/
npm install @capacitor/core @capacitor/cli @capacitor/android
npx cap init CloudSim com.example.cloudsim --web-dir=dist
npx cap add android
npx cap sync android
npx cap open android                           # build the APK in Android Studio
```

Release test plan

Area	Test IDs	How
End-to-end + class load	ST-01, ST-02	20 students build the same lab at once
Offline	ST-03, DT-05, SP-04	Install, switch to airplane mode, run labs and quizzes; watch the network for calls
Latency	SP-03, SP-06	100 validations; big topology; the on-device engine must also meet 200 ms
Install	DT-01, DT-02, DT-04	APK on several Android versions; PWA on major browsers; update keeps user data
Low-end devices	DT-03	Low-RAM phone: first launch, Pyodide load time, lab run
Learner acceptance	UAT-01 to UAT-05	Final UAT with students and a trainer
Gamification	1 test (badges)	Badge rules award the right badges

Tests

`pytest` - full suite **40 passed**. System, security and deployment tests are run by hand or with emulator scripts.

Definition of Done

- The whole app works in airplane mode after the first install, with no network calls.
- Badges are awarded correctly and progress survives closing the app.
- Trainer can import 20 student progress files and see one class table.
- Validation stays under 200 ms on a low-end device (or the limit is documented and accepted).
- APK and PWA pass every deployment test case; UAT feedback is addressed.

10.8 Stage Checklist Summary

Stage	Done when	Key files	Verified here
0	Tests run, API starts, DB exists, dev server opens	requirements.txt, pytest.ini, schema.sql	SQL not executed
1	CIDR checks, quiz, first lab work end to end	network.py, quiz.py, main.py, Canvas.jsx	Engine tests run; API wiring checked
2	Routing, firewall, IAM, compute, storage + generic checker	routing/firewall/iam/compute/storage/validator.py	Engine tests run
3	Three providers, cross-cloud peering, export/import	adapters/, topology.py	Engine tests run
4	Four specialty modules using the recipe	validator.py, new labs	Engine tests run
5	Offline PWA/APK, badges, class mode	engine.js, service-worker.js, gamification.py	Badge test run; JS syntax-checked only

11. Exam Preparation & Workflow Diagrams

This section gives learners a study path through the certifications in section 3, and gives developers/QA a quick-reference view of three flows already covered in detail elsewhere: bug handling (section 6), the VM lifecycle (compute.py, section 10.4) and a worked multi-cloud challenge lab (section 10.5).

11.1 Exam Preparation Roadmap

Level	Certifications
Fundamentals	AZ-900, AI-900, DP-900, SC-900
Associate	AZ-104, AZ-204, AZ-500, DP-203, DP-300, AZ-700
Expert	AZ-305, AZ-400, SC-100
Specialty	AZ-140, AZ-120, DP-420

Suggested study order: AZ-900 -> AZ-104 -> AZ-305 -> AZ-400 -> AZ-140. This is the fastest path from fundamentals to a specialty; AI-900, DP-900, SC-900, AZ-204, AZ-500, DP-203, DP-300, AZ-700, SC-100, AZ-120 and DP-420 branch off it depending on the role a learner is aiming for (security, data, DevOps).

This matches the platform's own roadmap: AZ-900 is Phase 1, AZ-104 is Phase 2, and AZ-305/AZ-400/AZ-140 are built in Phase 4 (section 10.6) once the AWS/GCP engine work from Phase 3 is in place.

11.2 Study Plan Checklist

A learner repeats this four-part loop for every module in the roadmap.

1. **Daily theory** - read that day's syllabus section and take notes (content/<module>/lessons.json).
2. **Quiz practice** - answer 10-15 MCQs on the material and review every wrong answer (quizzes/*.json, quiz.py).
3. **Lab simulation** - practise the matching skill hands-on: VM lifecycle, networking, storage or IAM (the labs in section 7 and 10).
4. **Review and revise** - a short daily recap, plus one mock test at the end of each week covering everything so far.

Step	Learner does	Platform feature used
Daily theory	Read lessons for today's topic	Theory Viewer, GET /api/modules/{code}/lessons
Quiz practice	10-15 MCQs, review mistakes	Quiz page, POST /api/quizzes/{id}/submit
Lab simulation	Build or fix a lab on the canvas	Canvas Editor, POST /api/labs/{id}/validate or /check
Review & revise	Recap notes; weekly mock test	Progress Dashboard, badges (gamification.py)

11.3 Bug Handling Flow

Every check in the Simulation Engine (CIDR, routing, firewall, IAM) follows the same four-step loop, already implemented as validator.run_checks() in section 10.4.

Bug handling flow

```
 [Detect]  ->  [Feedback]   ->    [Fix]     ->    [Validate]
Engine         Error /          Learner edits    Re-run the
validation     warning /        the config       same check:
(CIDR,         hint             (route, NSG,     pass -> badge
firewall,      message          IAM policy,      fail -> back
IAM)                            etc.)            to Feedback
```

Step	Example	Engine function
Detect	Two subnets overlap	network.find_issues()
Feedback	"web (10.0.1.0/24) overlaps app (10.0.1.128/25)"	network.ValidationError message
Fix	Learner changes app's CIDR to 10.0.2.0/24	learner action on the canvas
Validate	Re-run the check; 0 issues -> lab passed	validator.run_checks() / find_issues()

11.4 VM Lifecycle Flow

Implemented in engine/compute.py (section 10.4). A log entry is written on every transition, and the allowed transitions are fixed so an invalid move (for example stopping an already-stopped VM, CMP-04) is rejected.

VM lifecycle flow

```
[Create VM] -> [Running] -> [Stopped] -> [Deleted]
                    ^  |_____________|
                    |   (start/stop can repeat)
                    |
             Firewall / NSG rules are checked
             at every stage; a log entry is
             written on each state change.
```

From	Allowed next states	Notes
creating	running	First transition after create_instance()
running	stopped, deleted	CMP-02: stop writes a log entry
stopped	running, deleted	Can be restarted any number of times
deleted	(none)	Terminal state - CMP-04/05: any further action is rejected

11.5 Multi-Cloud Challenge Lab (Worked Example)

A concrete walk-through of the Azure <-> AWS <-> GCP connectivity lab from section 10.5 (multi-lab01/02), using non-overlapping ranges for all three clouds.

Cloud	Network	Address space
Azure	vnet-azure	10.0.0.0/16
AWS	vpc-aws	172.16.0.0/16
GCP	vpc-gcp	192.168.0.0/16

1. **CIDR check** - confirm none of the three ranges overlap (network.check_cidr_overlap, run pairwise).
2. **Route add** - peer Azure with AWS: routing.peer_networks() adds a reciprocal route on each side (10.0.0.0/16 on the AWS side, 172.16.0.0/16 on the Azure side). Repeat for the pair that needs to reach GCP.
3. **Firewall validate** - on GCP, add a firewall rule that allows inbound traffic from both 10.0.0.0/16 and 172.16.0.0/16 (firewall.evaluate() must return 'allow' for both source ranges).
4. **Connectivity test** - check reachability VM (Azure) -> EC2 (AWS) -> Compute Engine (GCP) with routing.can_reach() at each hop.

Worked example - CIDR + peering (uses the Stage 3 engine)

```
azure = Network("vnet-azure", "azure", "10.0.0.0/16")
aws   = Network("vpc-aws",   "aws",   "172.16.0.0/16")
gcp   = Network("vpc-gcp",   "gcp",   "192.168.0.0/16")

routes_azure, routes_aws, routes_gcp = [], [], []
assert peer_networks(azure, aws, routes_azure, routes_aws) == []
assert peer_networks(aws, gcp, routes_aws, routes_gcp) == []

gcp_rules = [
    Rule("allow-azure", 100, "inbound", "*", None, "10.0.0.0/16", "allow"),
    Rule("allow-aws",   100, "inbound", "*", None, "172.16.0.0/16", "allow"),
]
assert evaluate(gcp_rules, "inbound", "tcp", 443, "10.0.5.5")[0] == "allow"
assert can_reach(routes_azure, "172.16.4.4")   # Azure VM can reach the AWS side
```

This mirrors test IT-01 (connectivity), IT-02 (overlap refused) and IAM-03 (cross-cloud role conflicts) from section 7, and can be turned directly into lab file `multi-lab01` (section 10.5).

11.6 Challenge Lab Checklist

Check	Pass condition	Test ID
CIDR ranges valid	No two networks' address spaces overlap	IT-02
Route tables updated	Each peered network has a reciprocal route to the other	IT-01, NET-03
Firewall rules modified	The destination cloud allows traffic from both source ranges	NET-02, UT-13
Connectivity test passes	can_reach() succeeds for every required hop	IT-01, IT-03
Badge awarded	All checks above pass -> lab marked complete, badge shown	UC-15, gamification.py

12. Risks & Mitigations

Risk	Impact	Mitigation
CIDR overlap confusion	Learners cannot tell why a subnet or peering was rejected	network.find_issues() names both conflicting ranges in plain language (clear error messages) instead of a raw exception
Provider behaviour differs from real cloud	Learners get wrong mental model	Document assumptions; validate rules against official docs; review by cloud-certified reviewers
Large scope (many certifications)	Delays	Phase-wise release; start with AZ-900 and AZ-104
Low-end device performance	Poor experience, slow validation on cheap phones	Lightweight canvas, lazy loading, an optimised offline engine kept under the 200 ms latency budget, tested on low-end devices
IAM misconfiguration	A learner (or the engine) grants access that should be blocked	iam.evaluate() makes explicit deny always override allow, with implicit deny as the fallback, so a mistake fails closed, not open
Engine complexity grows	Bugs, slow validation	Isolated engine, strong unit tests, latency budget of 200 ms
Student data loss	A learner loses labs, quiz scores or badges	Everything is kept in local storage only, never billed or synced to a server; local persistence tests plus export/import of labs as backup
Vendor content changes	Outdated labs	Versioned lab definitions; periodic review

13. Final Note

This document combines the complete process in one place: Vision, Architecture, Theory, Practical simulation, Use cases, Testing stages, Test cases, Roadmap, the stage-by-stage Implementation Guide and the Exam Preparation workflow diagrams. It is ready to be used directly by the developer and QA teams. Items such as detailed API schemas, exact device matrix and full quiz banks can be added as the project moves from phase to phase.

For trainers and students: every part of this platform - theory, quizzes, labs and challenges - runs completely offline once installed, costs nothing, and never touches a real cloud account or bill. Mistakes are safe to make: the simulation engine catches them (CIDR overlaps, blocked routes, over-permissive IAM) and explains what to fix, so learners can practise Azure, AWS and GCP skills freely in a sandbox before ever touching production infrastructure.